

seq2seq

implemented by scratch

Leonardo Angellotti

Dataset

eng-span

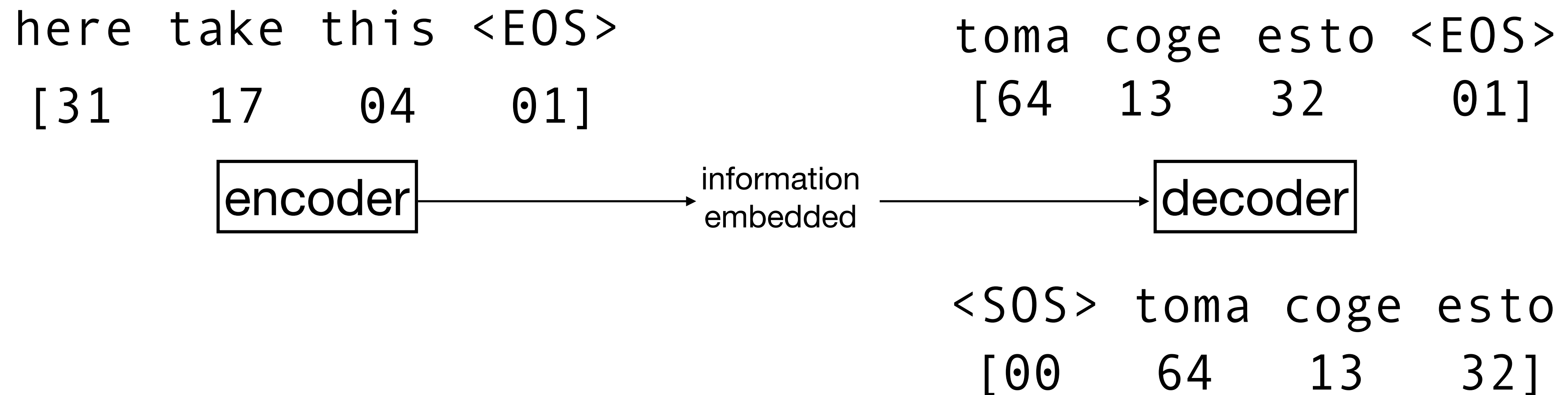
Go. Ve.
Go. Vete.
Go. Vaya.
Go. Váyase.
Hi. Hola.
Run! ¡Corre!
Run. Corred.
Who? ¿Quién?
Fire! ¡Fuego!
Fire! ¡Incendio!
Fire! ¡Disparad!
Help! ¡Ayuda!
Help! ¡Socorro! ¡Auxilio!
Help! ¡Auxilio!
Jump! ¡Salta!
Jump. Salte.
Stop! ¡Parad!
Stop! ¡Para!
Stop! ¡Pare!
Wait! ¡Espera!
Wait. Esperen.
Go on. Continúa.
Go on. Continúe.

seq2seq model

A Recurrent Neural Network (RNN) processes data sequentially, feeding the output from one step back into the network as input for the next step.

sequence-to-sequence (seq2seq) model removes constraints on input and output length and alignment.

This flexibility makes seq2seq models particularly well suited for tasks such as machine translation.



Language vocabulary class

the cat is on the table
i like pizza
the pizza is on the table
the cat likes pizza

word to index: {'the': 2, 'cat': 3, 'is': 4, 'on': 5, 'table': 6, 'i': 7, 'like': 8,
 'pizza': 9, 'likes': 10}

index to word: {0: 'SOS', 1: 'EOS', 2: 'the', 3: 'cat', 4: 'is', 5: 'on', 6: 'table',
 7: 'i', 8: 'like', 9: 'pizza', 10: 'likes'}

number of words: 11

One-hot encoding

```
one_hot_word("pizza", lang)
```

```
tensor([0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0.]
```

```
encoded = one_hot_sentence("the cat likes pizza", lang)
```

```
encoded sentence:
```

```
tensor([[0., 0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],  
        [0., 0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],  
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 1.],  
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 1., 0.]])
```

```
shape: torch.Size([4, 11])
```

'the'	index=2	count=5
'cat'	index=3	count=2
'is'	index=4	count=2
'on'	index=5	count=2
'table'	index=6	count=2
'i'	index=7	count=1
'like'	index=8	count=1
'pizza'	index=9	count=3
'likes'	index=10	count=1

Normalize string

Lowercases the text,
Cleans up spacing,
Normalizes Unicode characters,
Separates punctuation,
Removes unwanted characters

```
text = "Hélló!!!   How's it going? 😊"
```

```
print(normalize_w(text))
```

```
hélló ! ! ! how s it going ?
```

filter_pairs

(optionally)

filter **long sentence pairs** so the dataset contains only sentences within a fixed maximum length

```
filter_pairs(pairs)[::200][:10]

[['go .', 've .'],
 ['back off .', 'apártate .'],
 ['warn tom .', 'avísale a tom .'],
 ['is tom ok ?', 'está bien tom ?'],
 ['tom s sad .', 'tom está triste .'],
 ['i can sing .', 'puedo cantar .'],
 ['look at us .', 'míranos .'],
 ['we ll help .', 'ayudaremos .'],
 ['here we are !', 'aquí estamos !'],
 ['let me help .', 'déjame ayudar .']]
```


using batch size

```
input_lang, output_lang, pairs = read_pair('eng', 'span')
```

118964 sentence pairs readed

reduced to 7256 sentence pairs ← using filter

Counted words:

eng word counted: 2235

span word counted: 4082

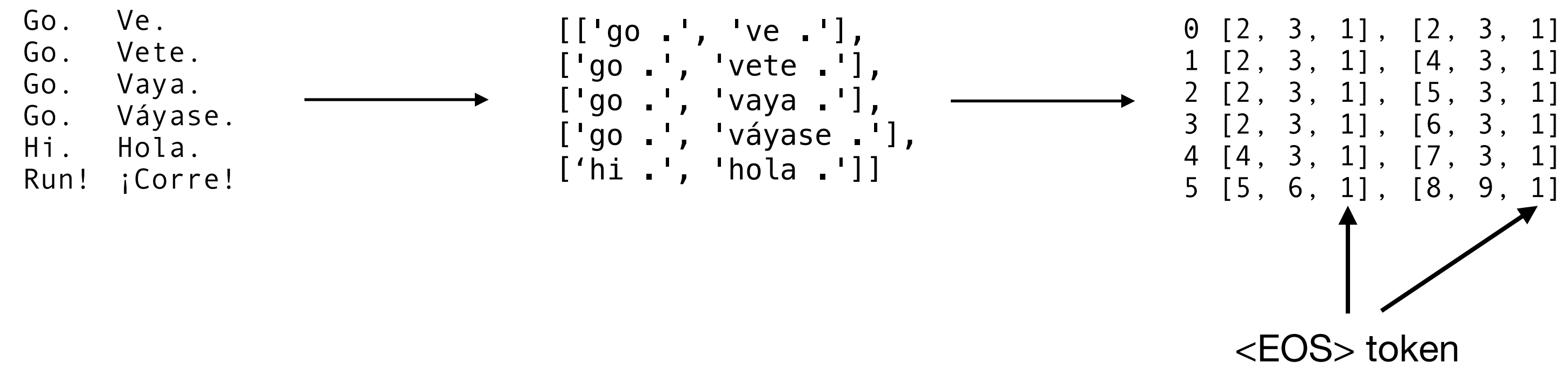
['close the blinds .', 'cierra las persianas .']

using a batch size

wrap into

```
input_lang, output_lang, train_data_loader = get_data_loader(batch_size=32, 'eng', 'span')
```

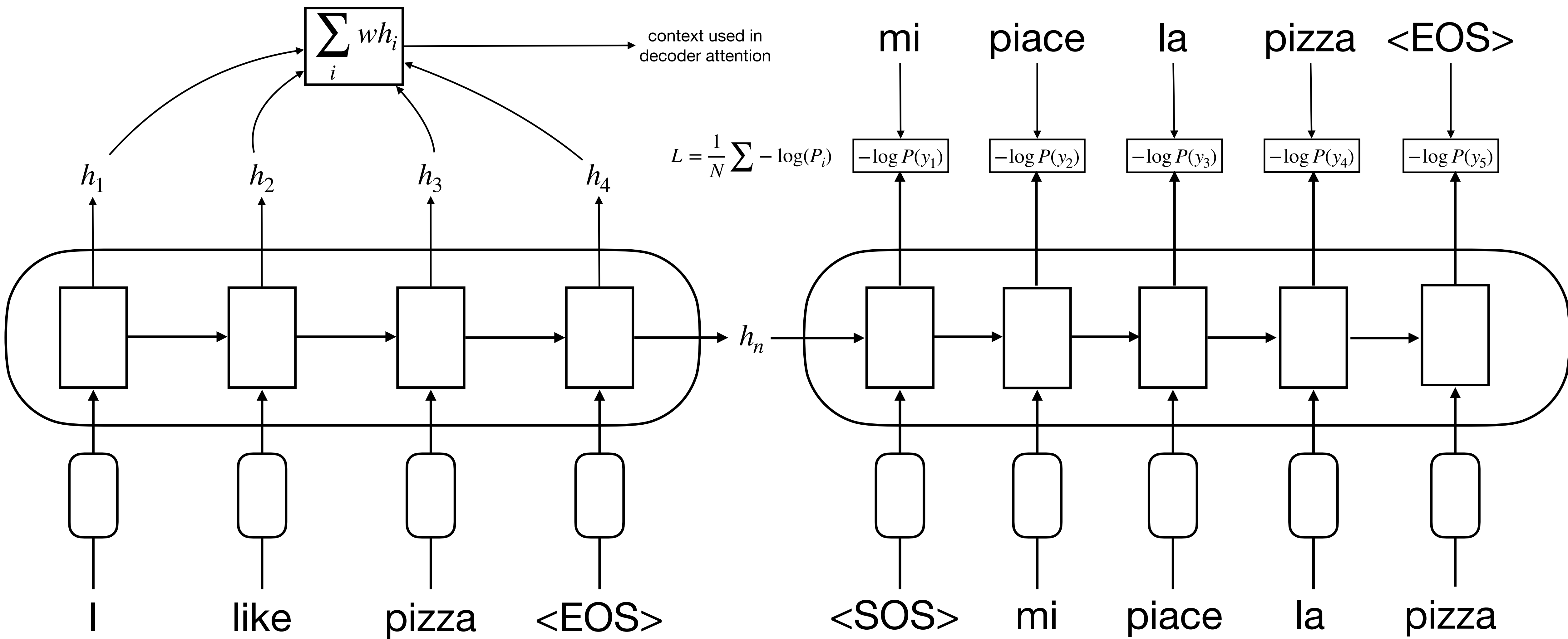
get_data_loader



Train

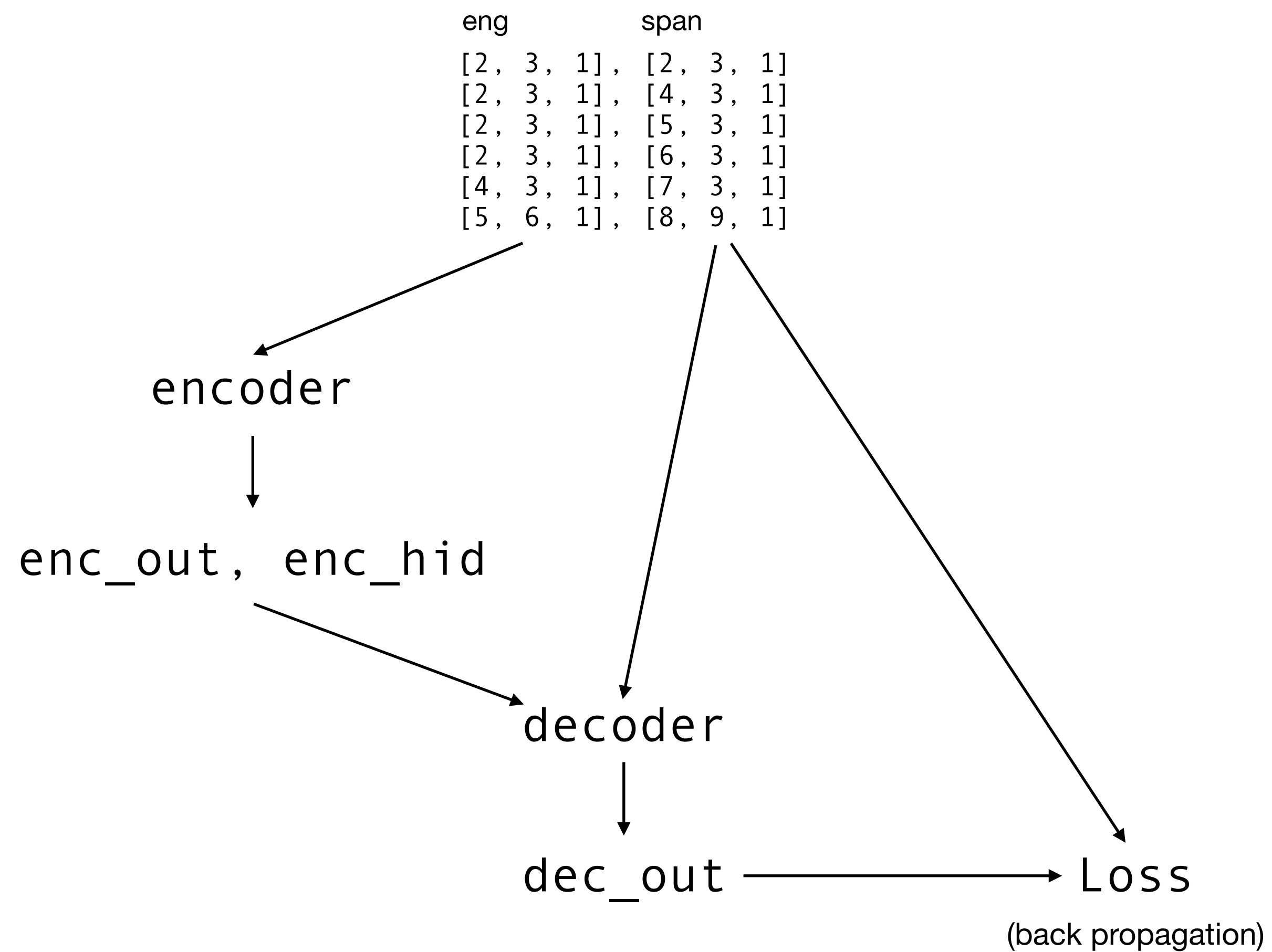
For n_epochs:

single_epoch



single_epoch

For data in dataloader:



	sentence length							
vocabulary dim	0.10	0.20	0.15	0.05	0.30	0.10	0.25	0.10
	0.20	0.10	0.10	0.15	0.10	0.20	0.15	0.20
	0.15	0.15	0.20	0.10	0.15	0.15	0.10	0.15
	0.25	0.20	0.15	0.20	0.10	0.15	0.20	0.15
	0.20	0.25	0.25	0.30	0.20	0.25	0.20	0.25
	0.10	0.10	0.15	0.20	0.15	0.15	0.10	0.15
	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8

dec_out

0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0
1	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	1

target

Loss

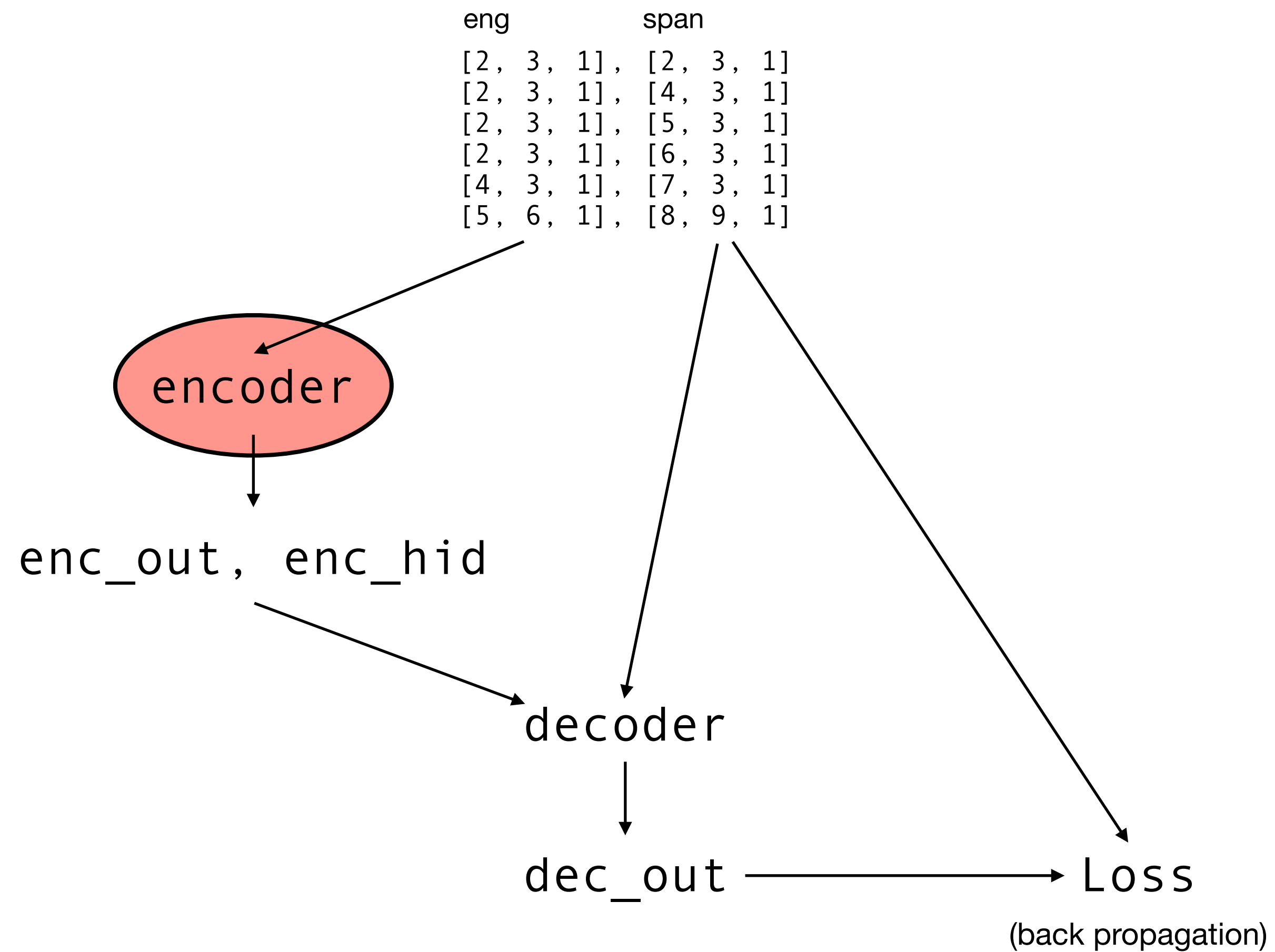
$$\mathcal{L}_{\text{NLL}} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i)$$

[0 , 0 , 0 . 2 , 0 . 4 , 0 . 1 , 0 , 0 . 3]	predicted token
[0 , 0 , 1 , 0 , 0 , 0 , 0]	target

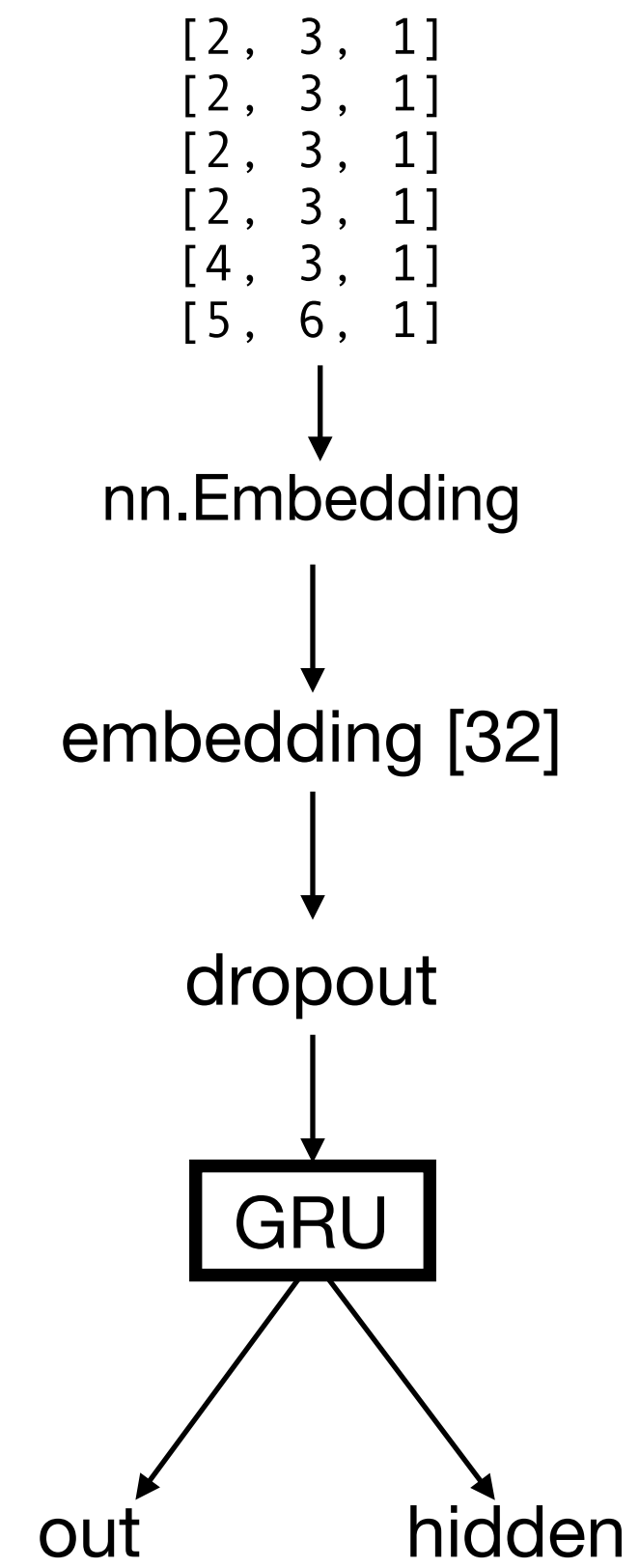
$$-\log(0.2) \approx 1.609$$

single_epoch

For data in dataloader:



encoder

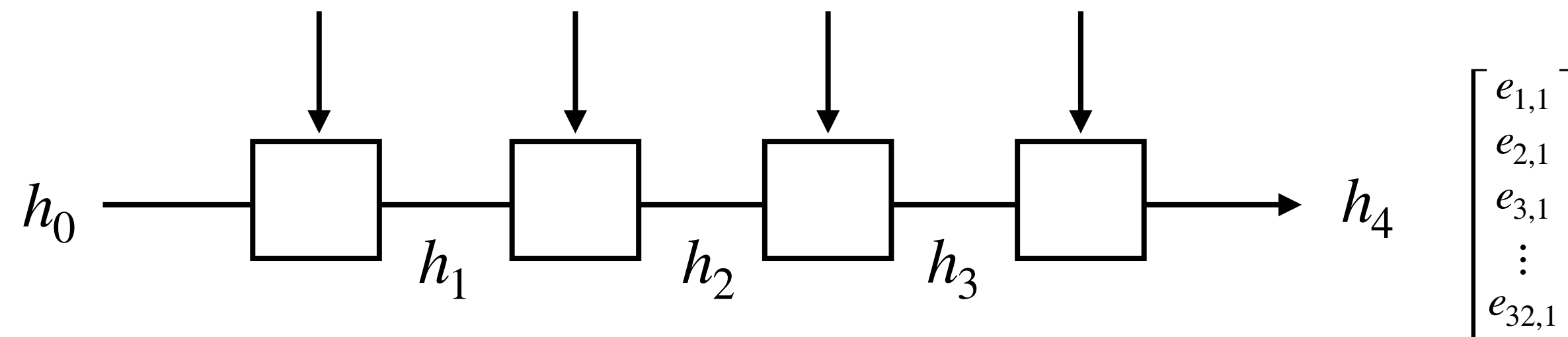


here take this <EOS>

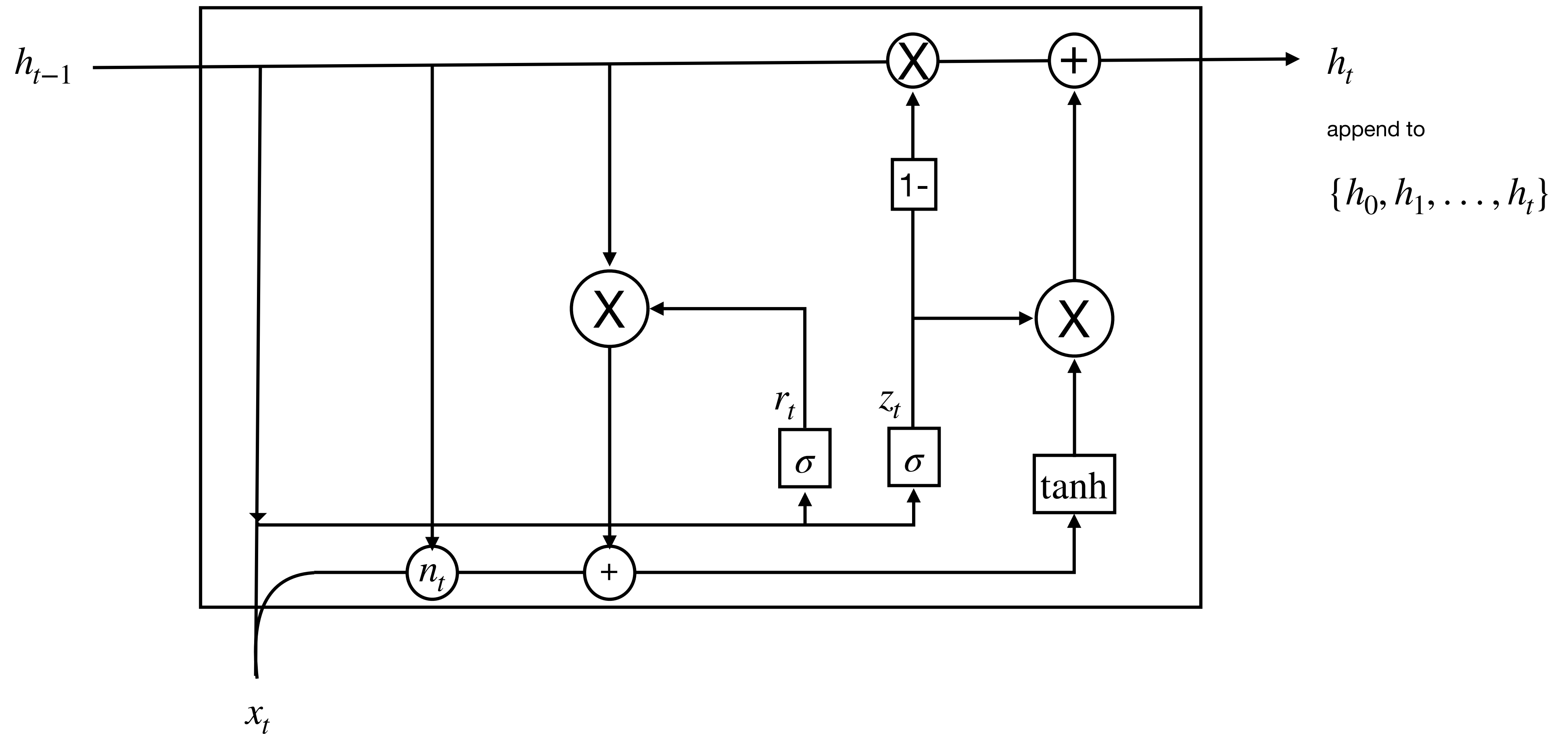
[31 17 04 01]

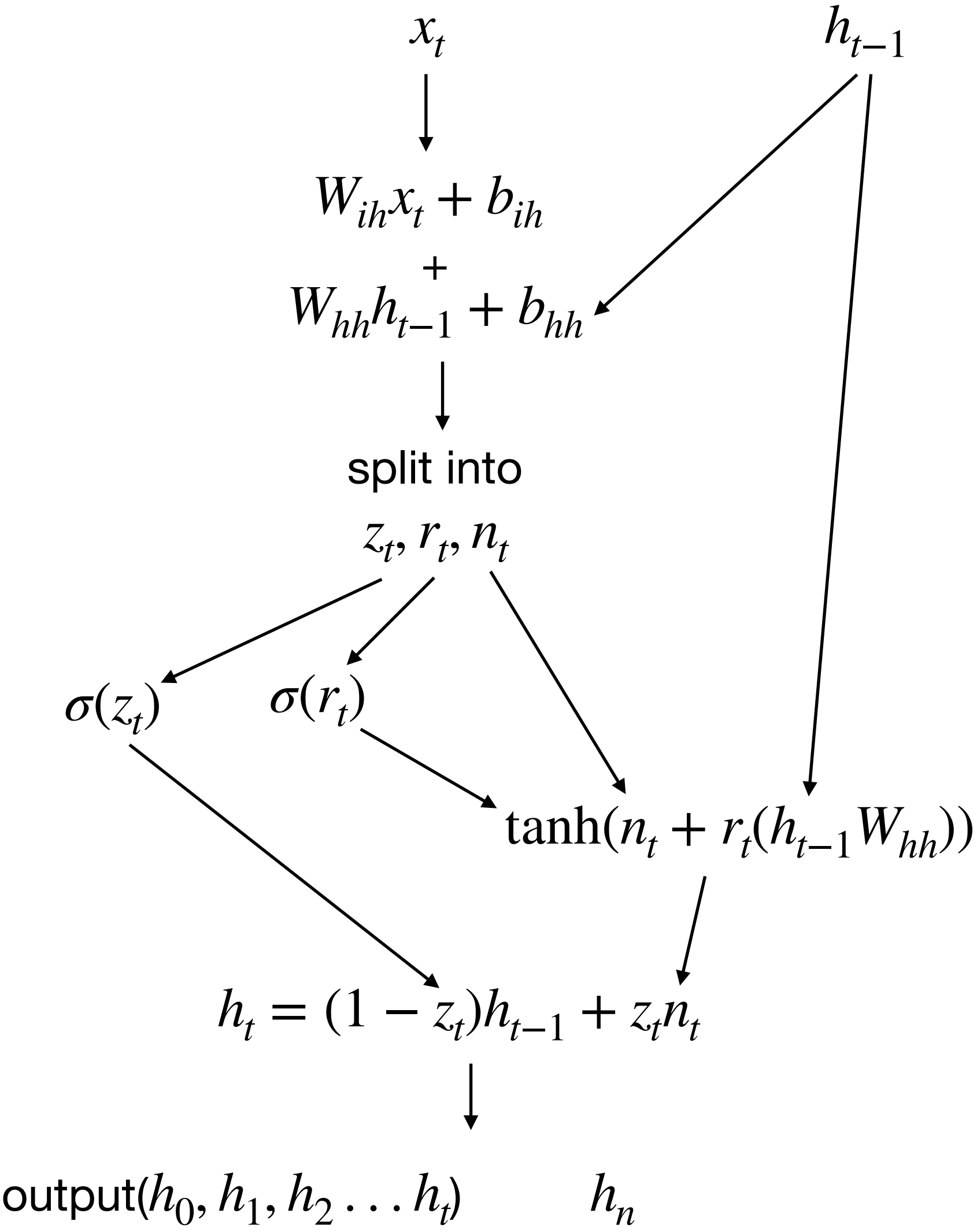
nn.Embedding

$$\mathbf{E} \in \mathbb{R}^{32 \times 4} = \begin{bmatrix} e_{1,1} & e_{1,2} & e_{1,3} & e_{1,4} \\ e_{2,1} & e_{2,2} & e_{2,3} & e_{2,4} \\ e_{3,1} & e_{3,2} & e_{3,3} & e_{3,4} \\ \vdots & \vdots & \vdots & \vdots \\ e_{32,1} & e_{32,2} & e_{32,3} & e_{32,4} \end{bmatrix}$$

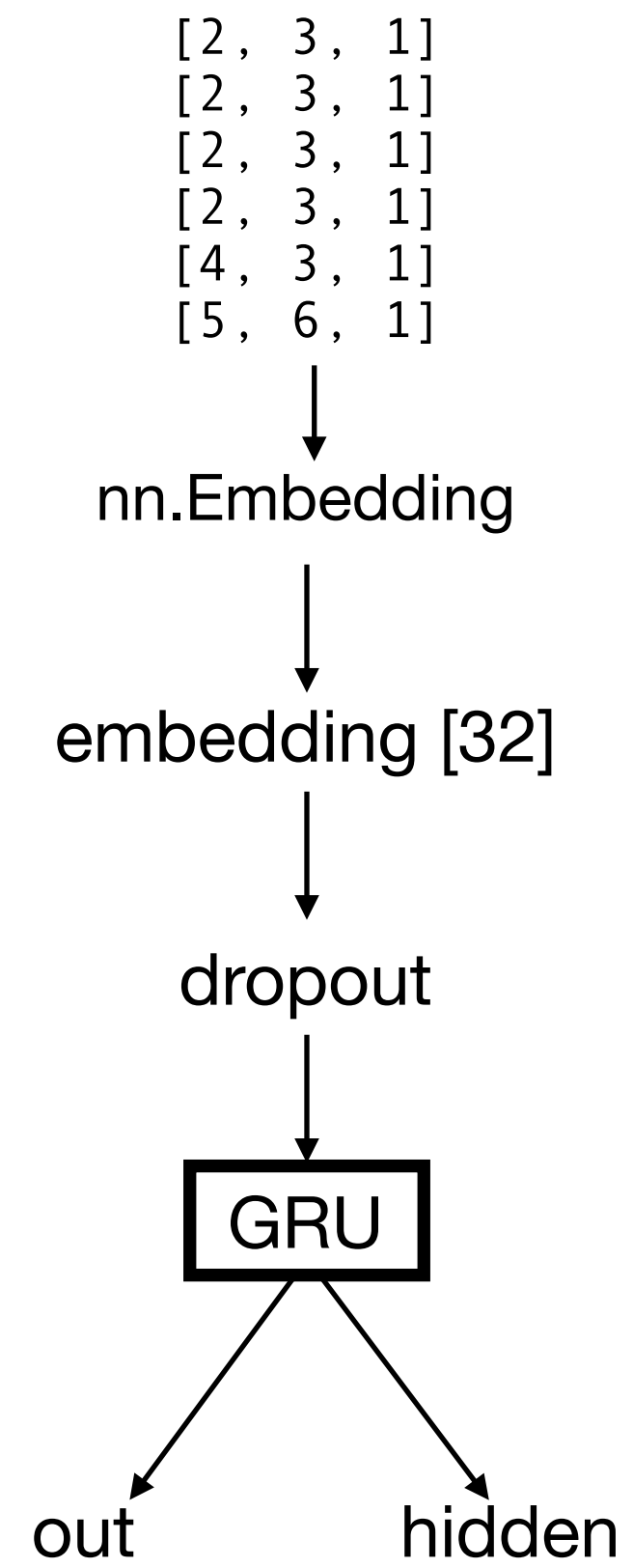


GRU unit



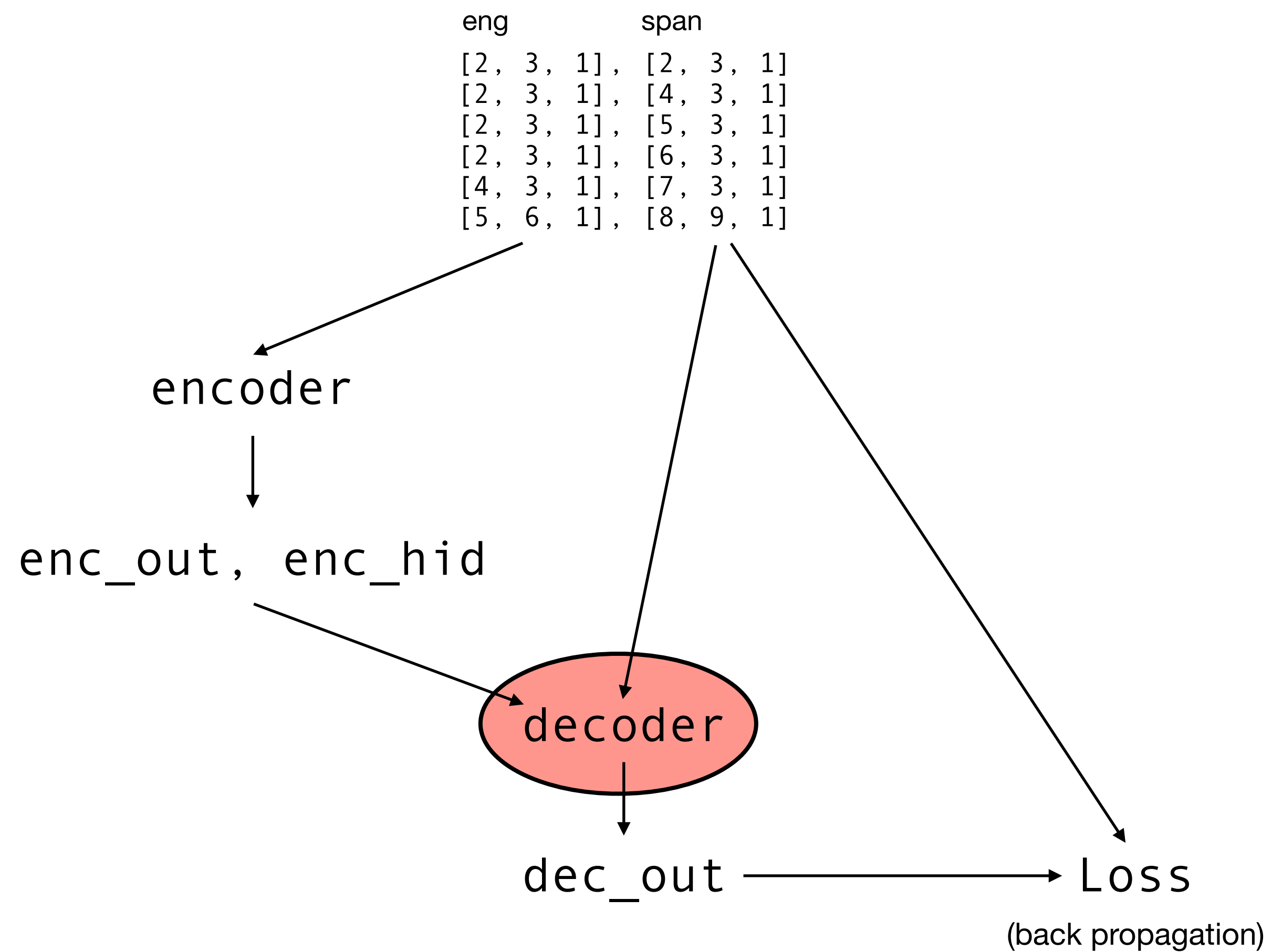


encoder



single_epoch

For data in dataloader:



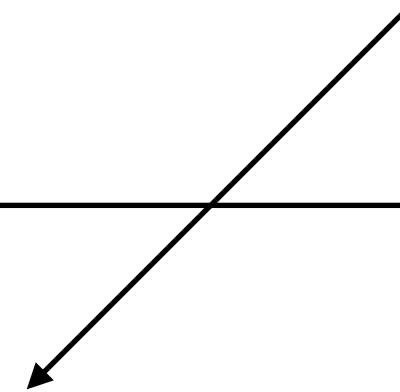
decoder

(Target string)

[2, 3, 1]
[4, 3, 1]
[5, 3, 1]
[6, 3, 1]
[7, 3, 1]
[8, 9, 1]

$\text{enc_output}(h_0, h_1, h_2 \dots h_t)$

h_n



$\text{decoder_in} = [\text{<SOS>}]$

$\text{decoder_h} = h_n$ (final encoder state)

$\text{decoder_out} = []$

$\text{attn} = []$

For max_length :

$\text{forward_step}(\text{decoder_in}, \text{decoder_h}, \text{enc_output}(h_0, h_1, h_2 \dots h_t))$

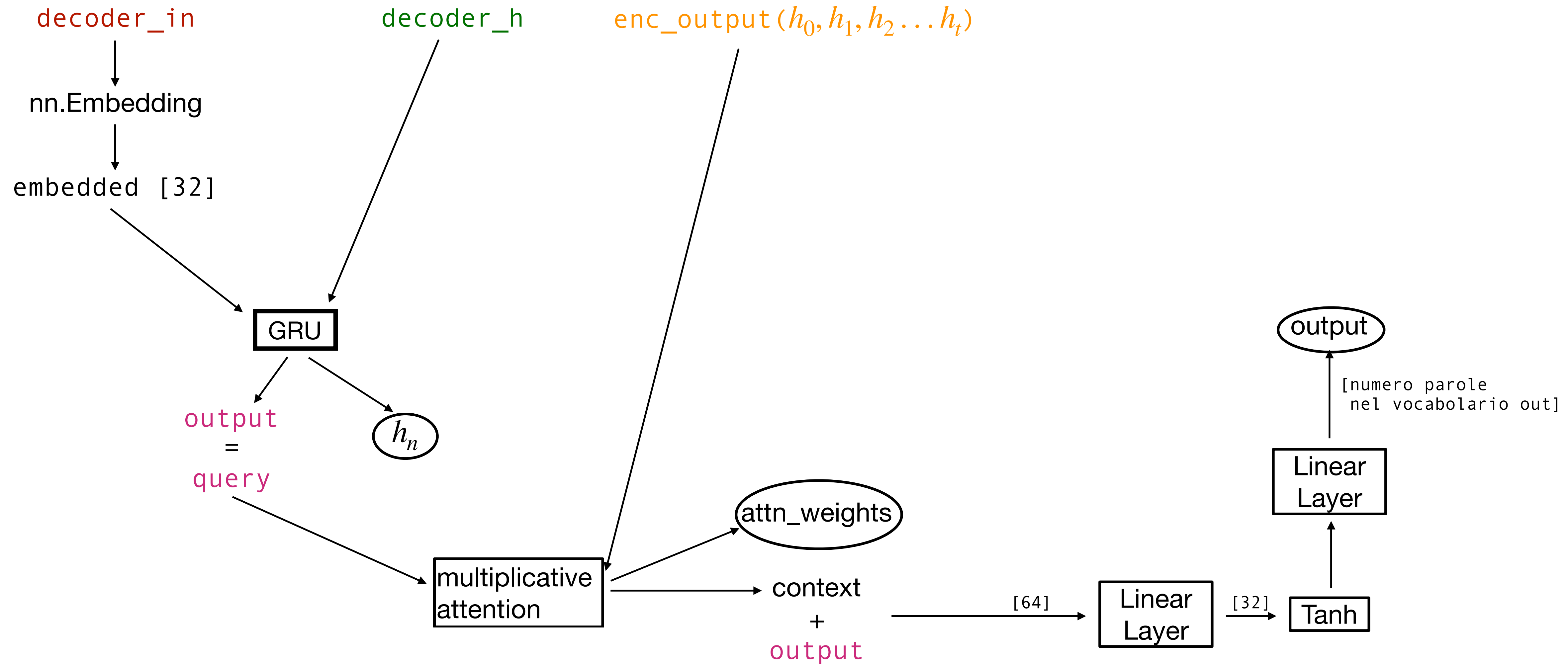
The additive and multiplicative score attention is computed in different ways,

Additive attention chooses what to focus on before updating the decoder

multiplicative attention combines the focus after the decoder produces its state

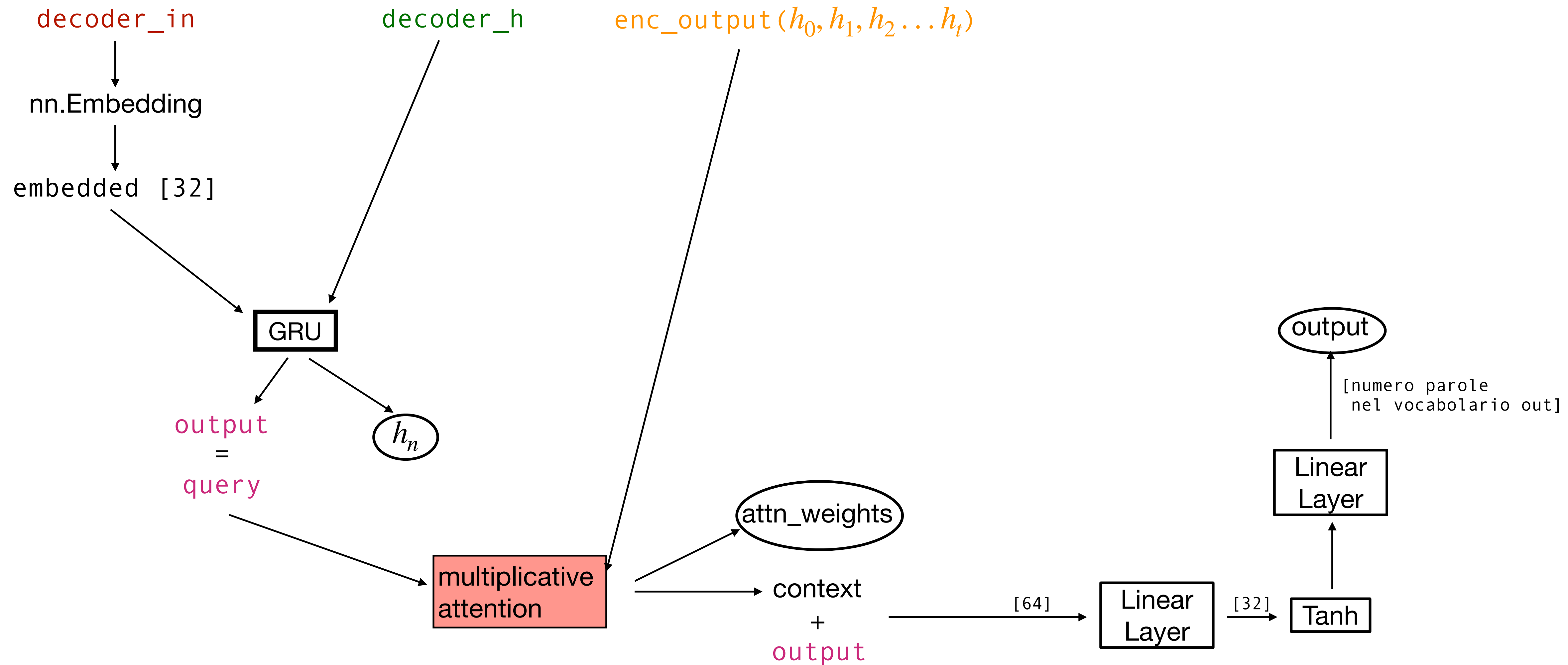
foward_step

multiplicative attention

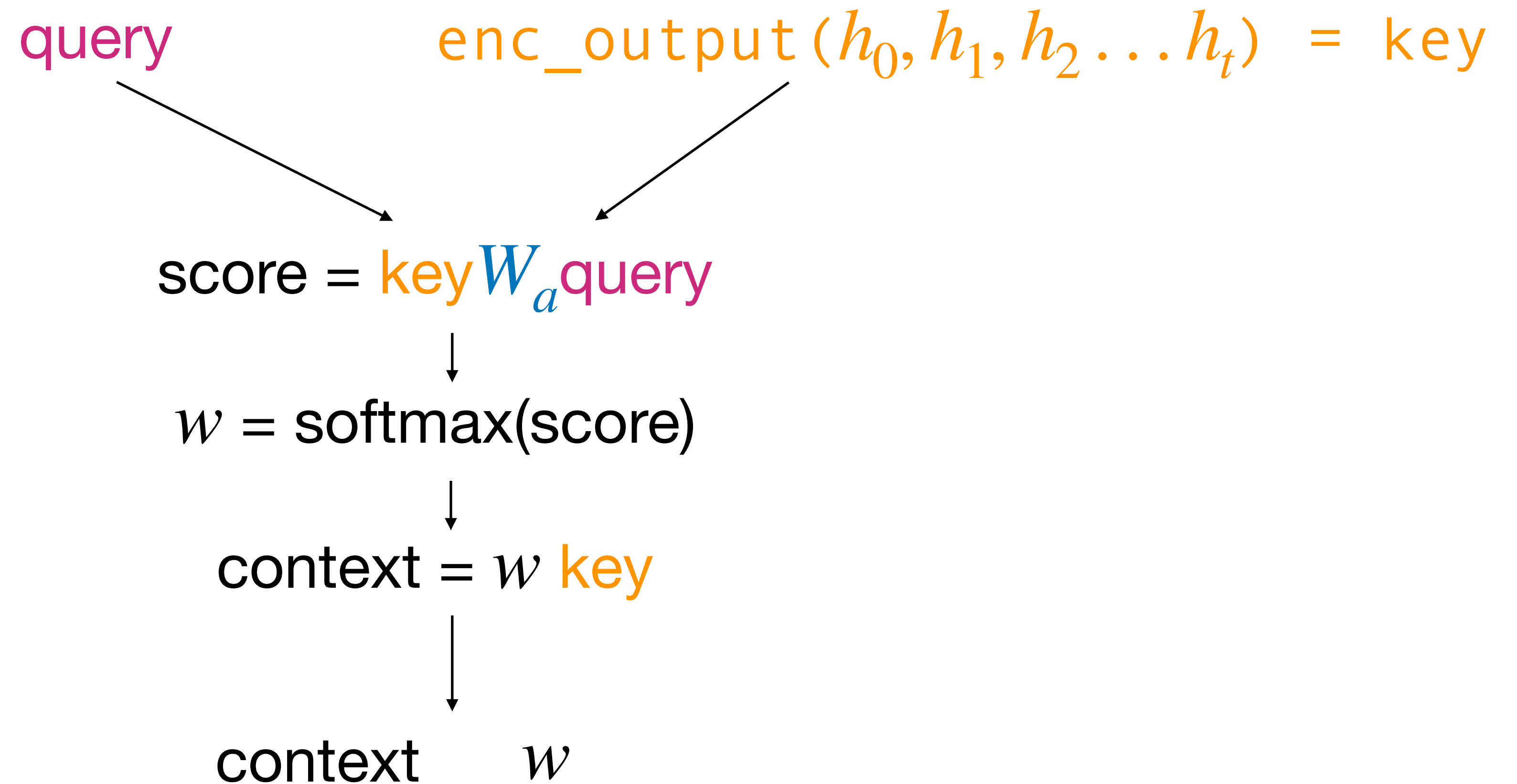


foward_step

multiplicative attention

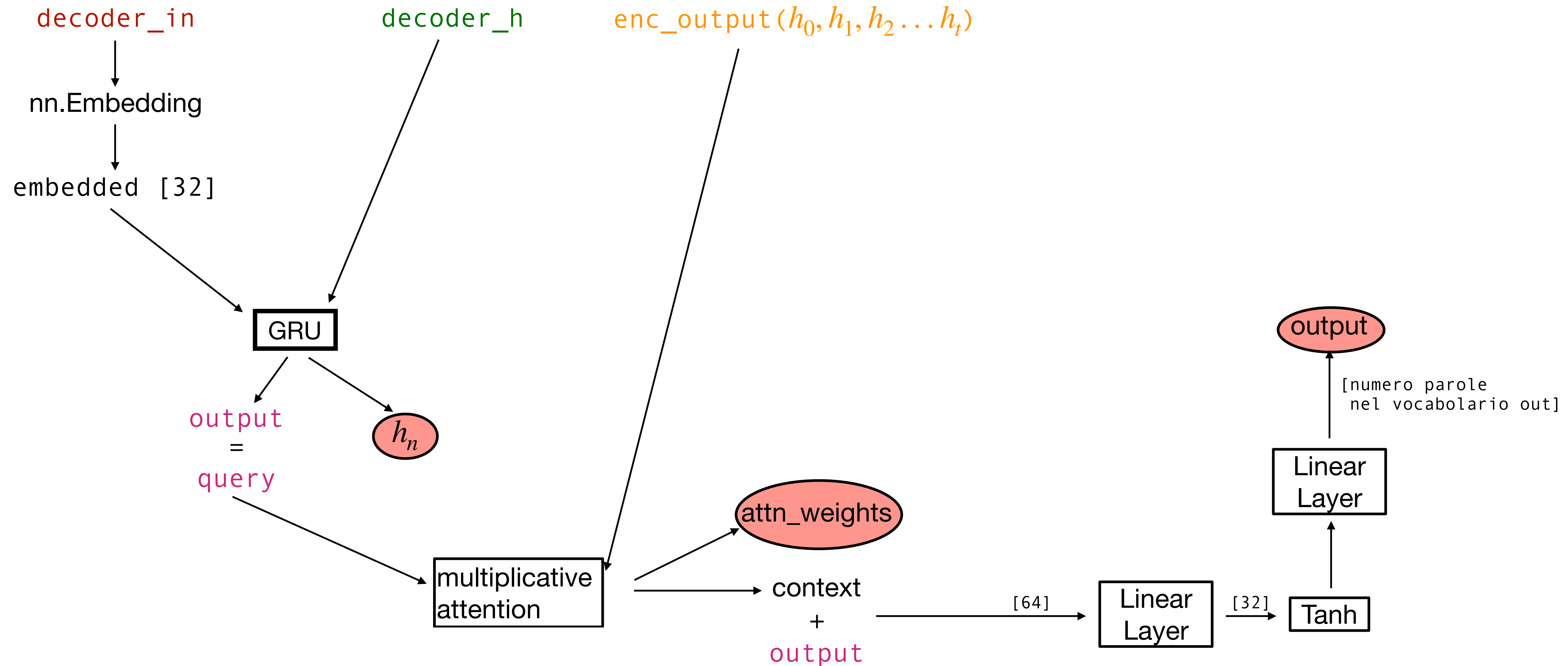


multiplicative attention



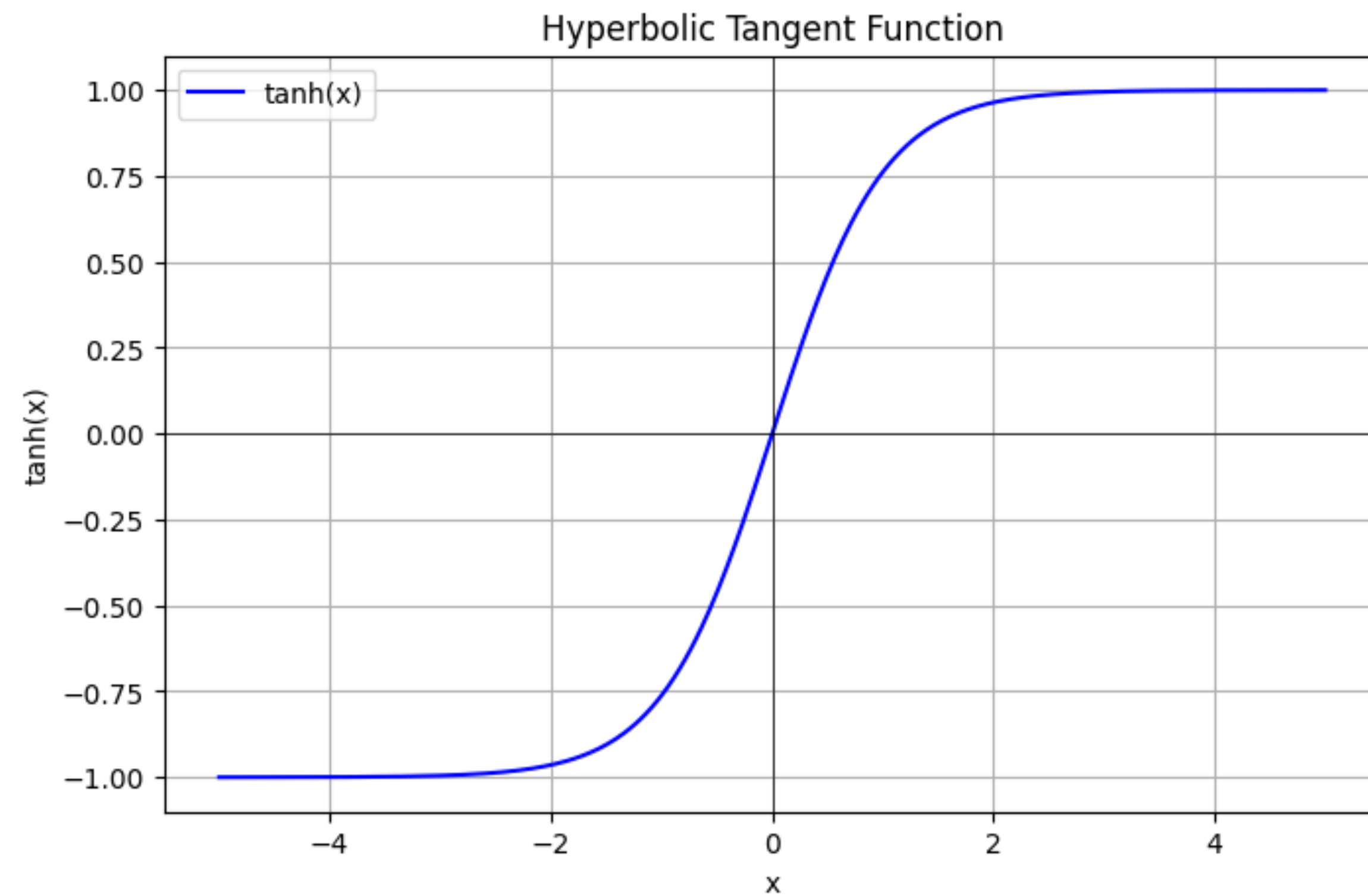
foward_step

multiplicative attention



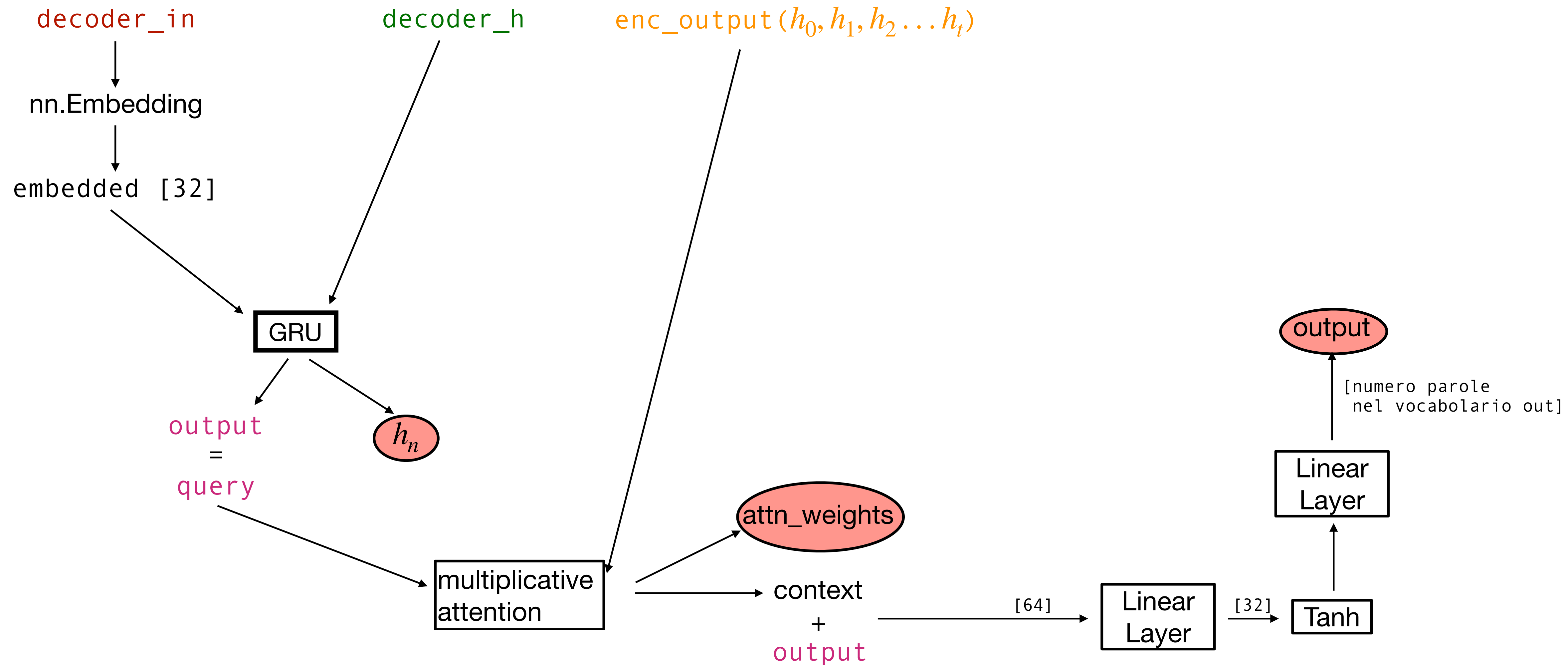
Tanh

$-\infty, +\infty$



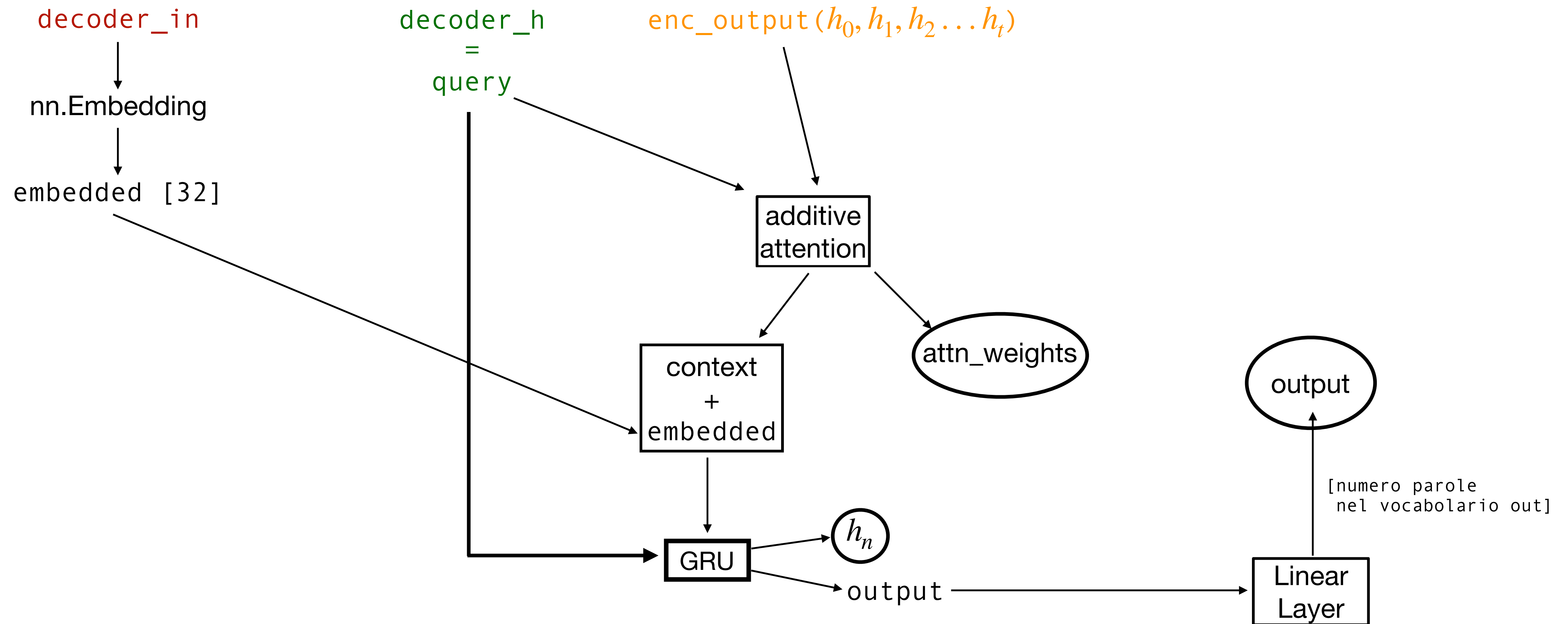
$(-1, +1)$

foward_step multiplicative attention



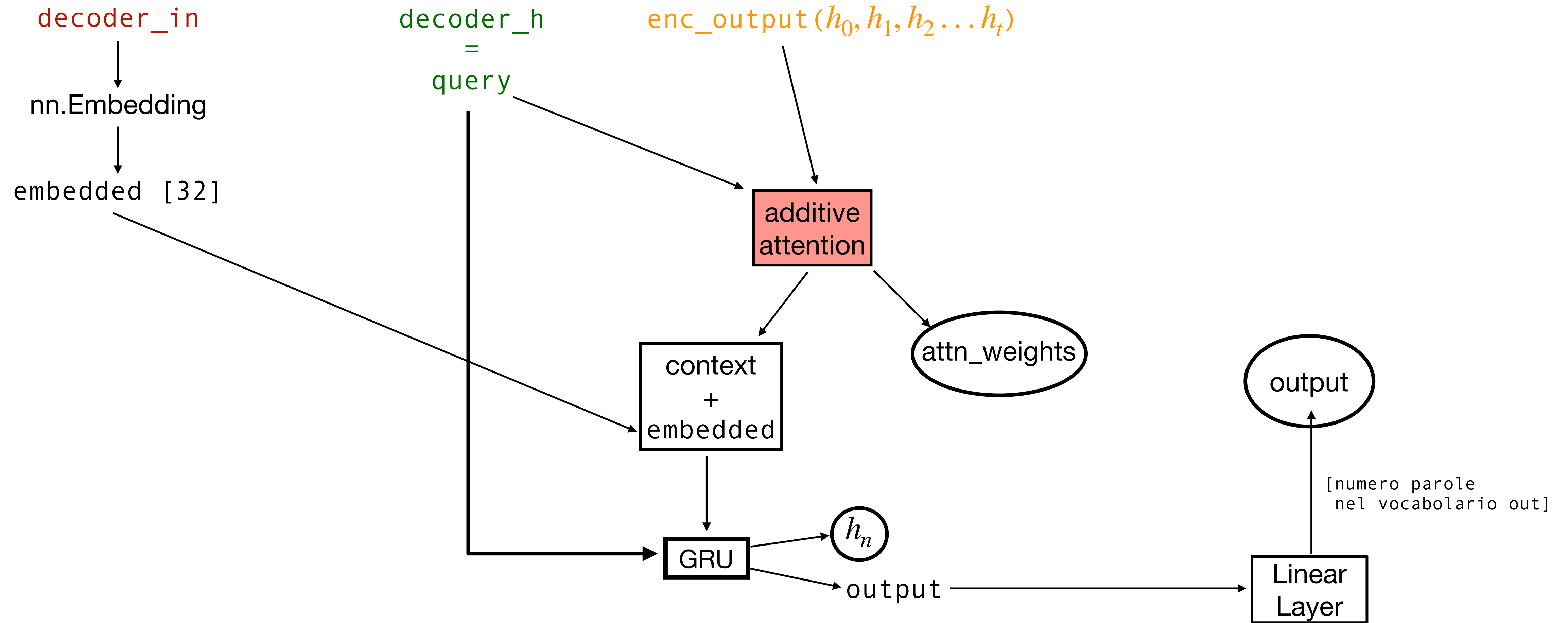
foward_step

Additive attention



foward_step

Additive attention



additive attention

$$\text{scores} = V_a(\tanh(W_a \text{query} + U_a \text{key}))$$



$$w = \text{softmax}(\text{scores})$$



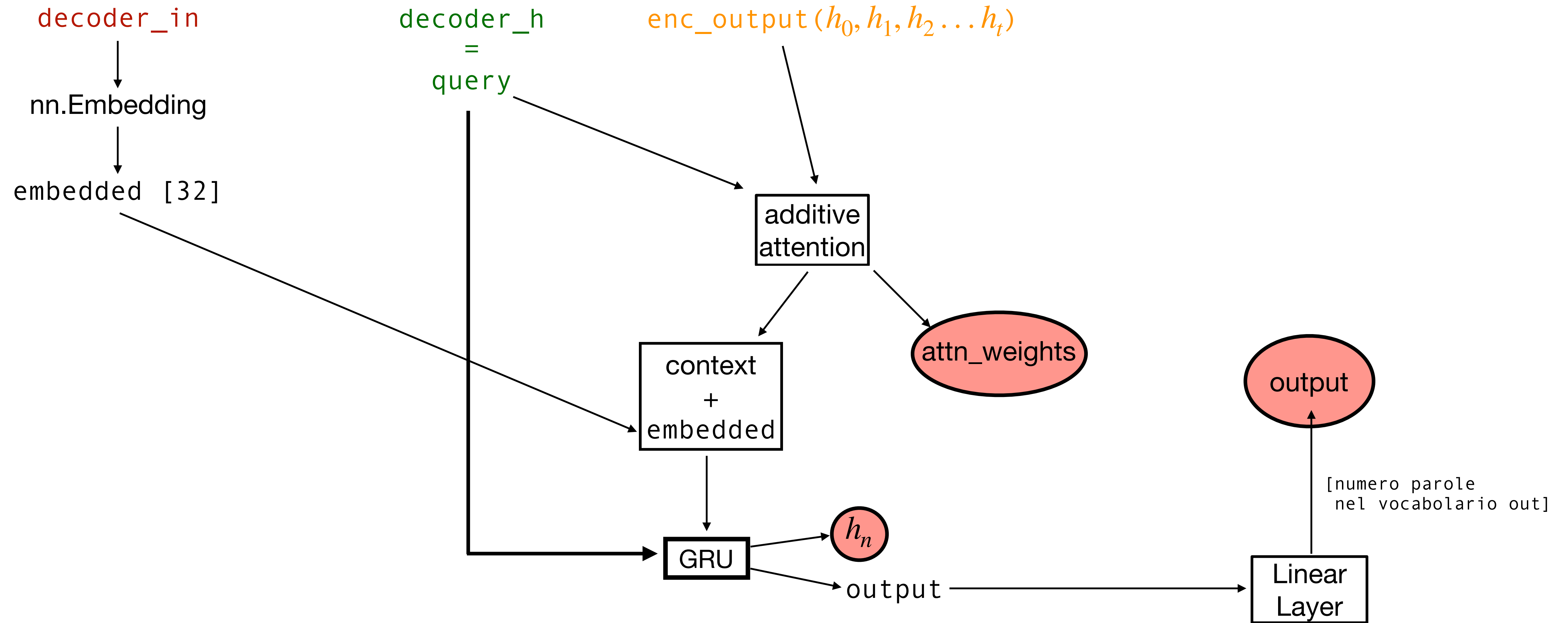
$$\text{context} = w \text{key}$$



context w

foward_step

Additive attention



decoder

For max_lenght:

foward_step(decoder_in, decoder_ h_0 , enc_output($h_0, h_1, h_2 \dots h_t$))

output, decoder_ h_n , w_{attn}

decoder_out.append(output)

attn.append(w_{attn})

(Target string)

decoder_in =

2	3	1
4	3	1
5	3	1
6	3	1
7	3	1
8	9	1

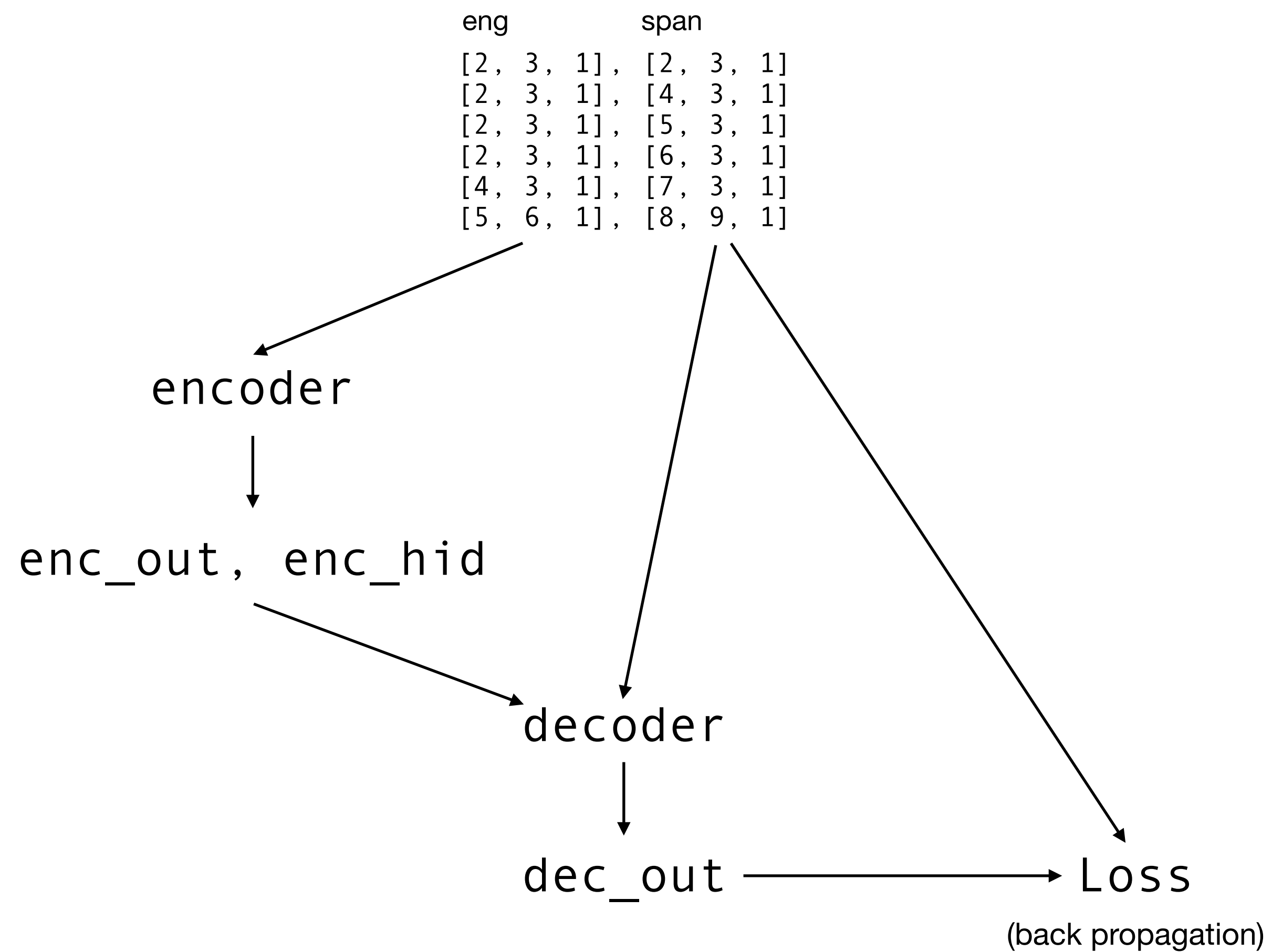
i

log_softmax(decoder_out)

decoder_out, decoder_ h_n , attn

single_epoch

For data in dataloader:



Train

For n_epochs:

single_epoch

optimizer

```
enc_opt = optim.Adam(encoder.parameters(), lr=0.001)
```

```
dec_opt = optim.Adam(decoder.parameters(), lr=0.001)
```

parameters to train

(Additive)

Decoder parameters:

```
embedding.weight 130624
attention.Wa.weight 1024
attention.Wa.bias 32
attention.Ua.weight 1024
attention.Ua.bias 32
attention.Va.weight 32
attention.Va.bias 1
gru.W_ih 6144
gru.W_hh 3072
gru.b_ih 96
gru.b_hh 96
out.weight 130624
out.bias 4082
```

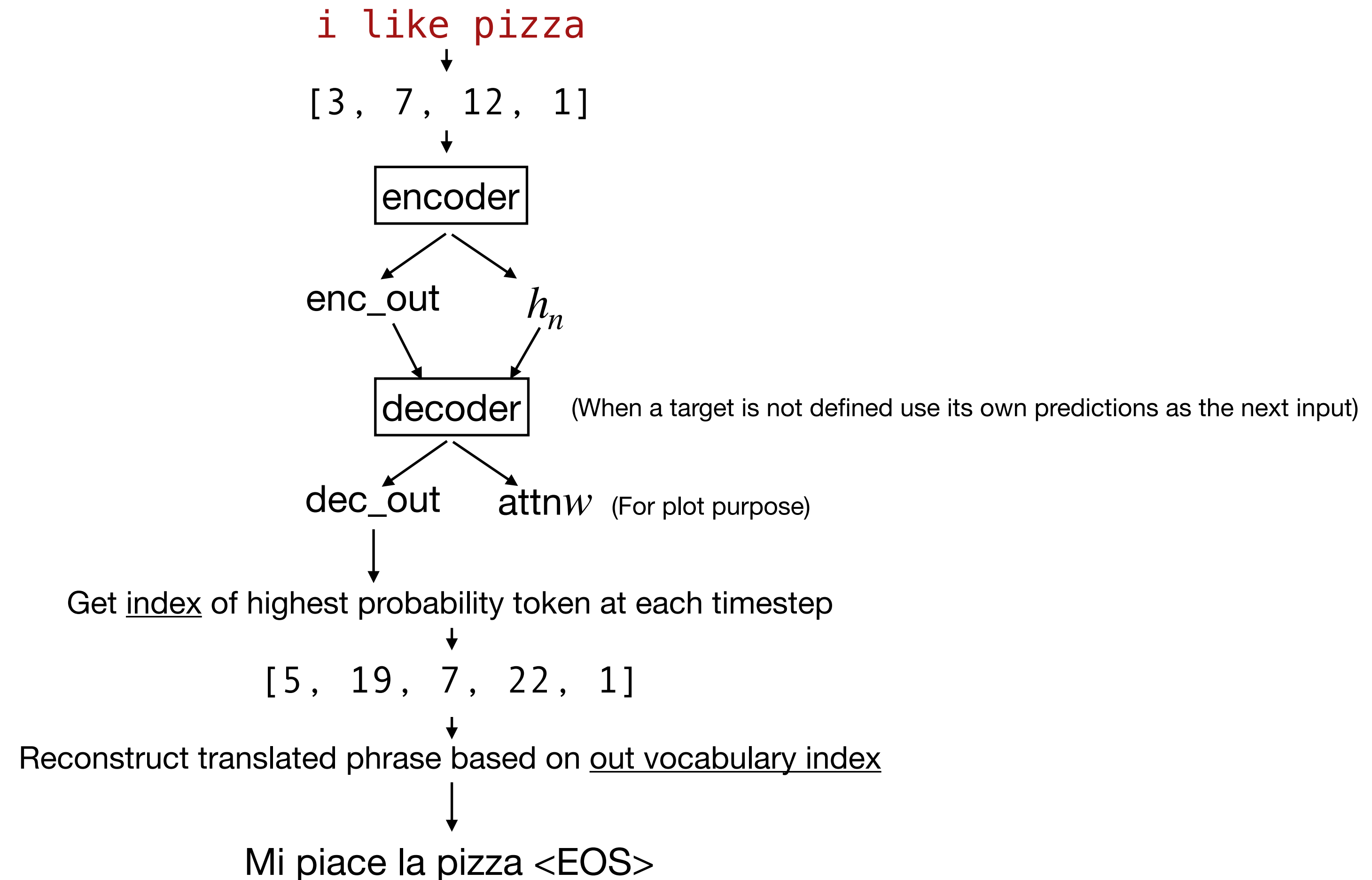
Encoder parameters:

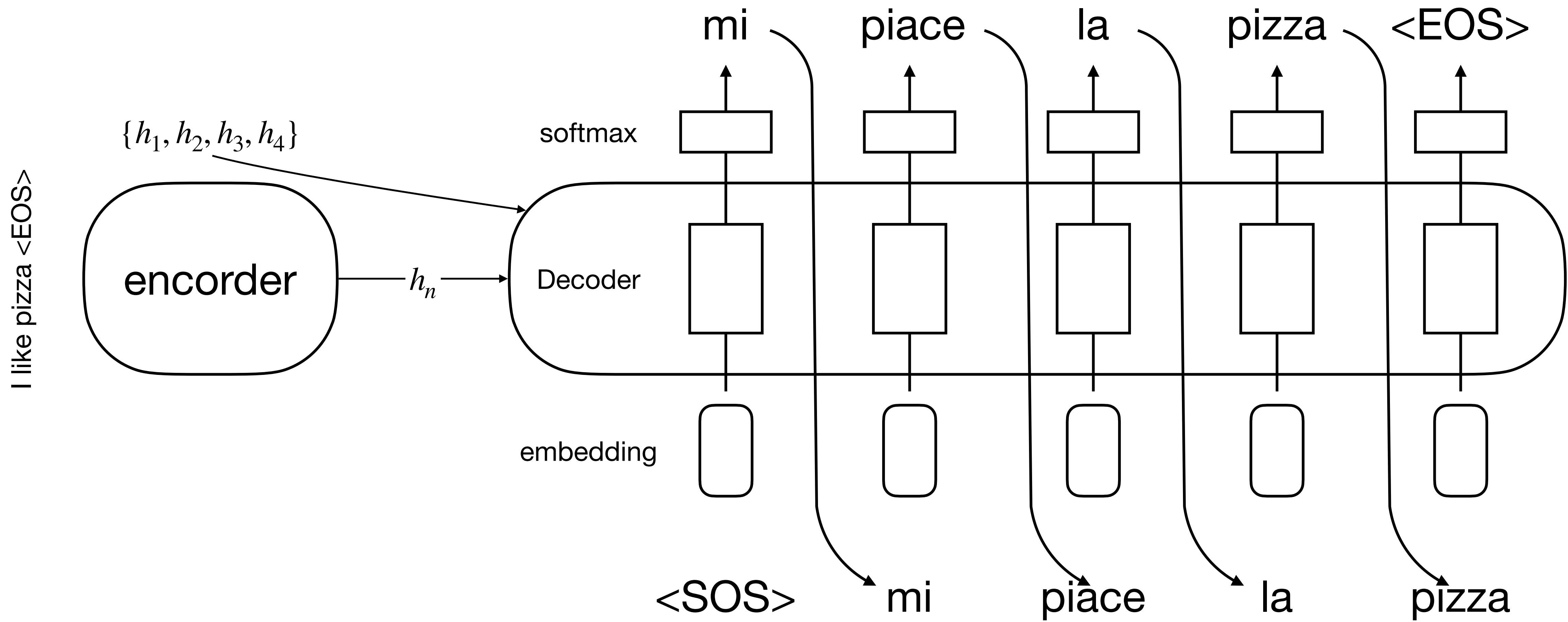
```
embedding.weight 71520
gru.W_ih 3072
gru.W_hh 3072
gru.b_ih 96
gru.b_hh 96
```

Encoder parameters: 77856
Decoder parameters: 276883
Total parameters: 354739

evaluation

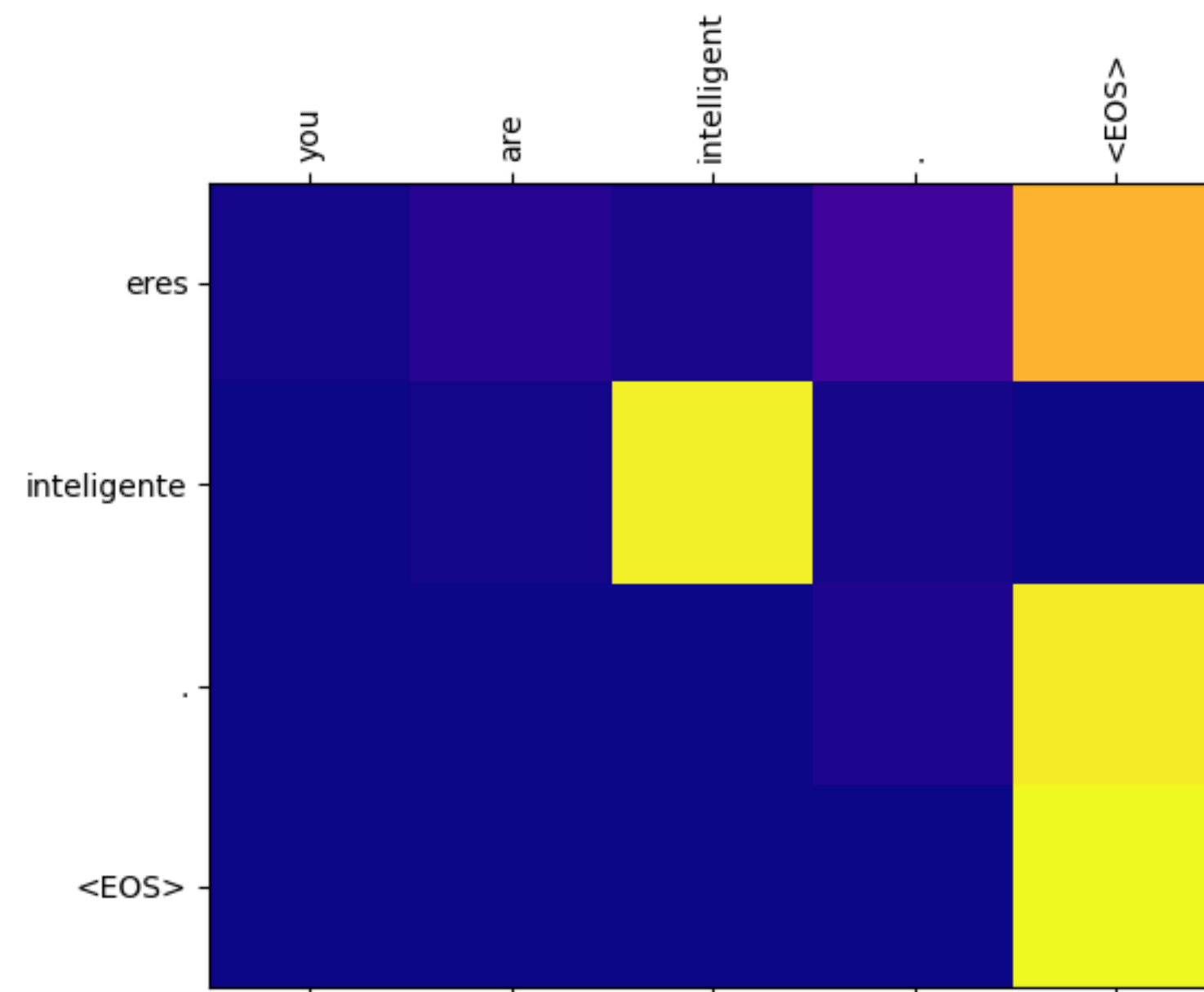
Disable gradient computation for evaluation



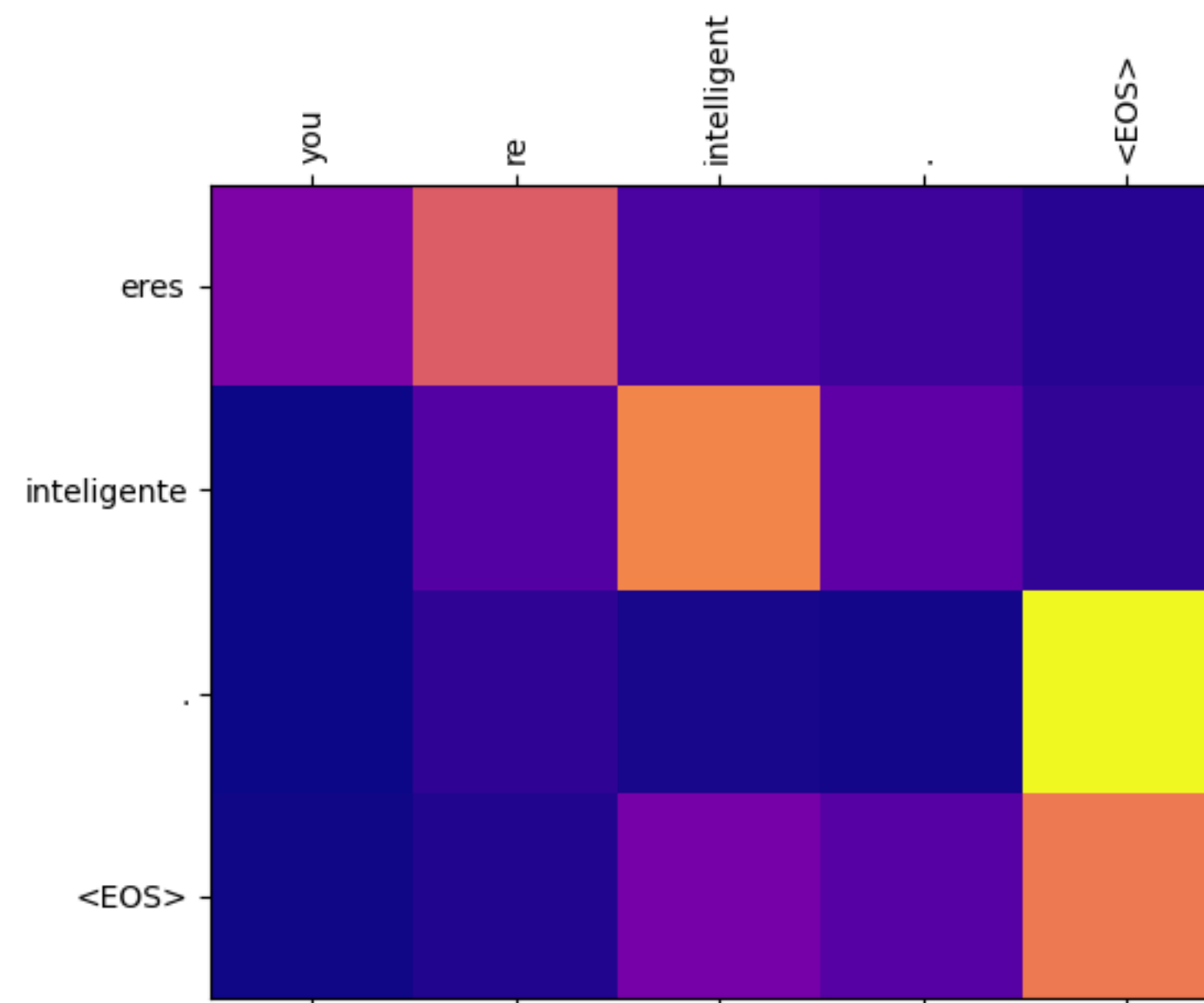


Visualizing attention

One of the most valuable aspects of the attention mechanism is that its outputs are highly interpretable. Since attention assigns weights to specific encoder outputs for each input token, we can see which parts of the input the network is focusing on at each decoding step.



Multiplicative



Additive

Results

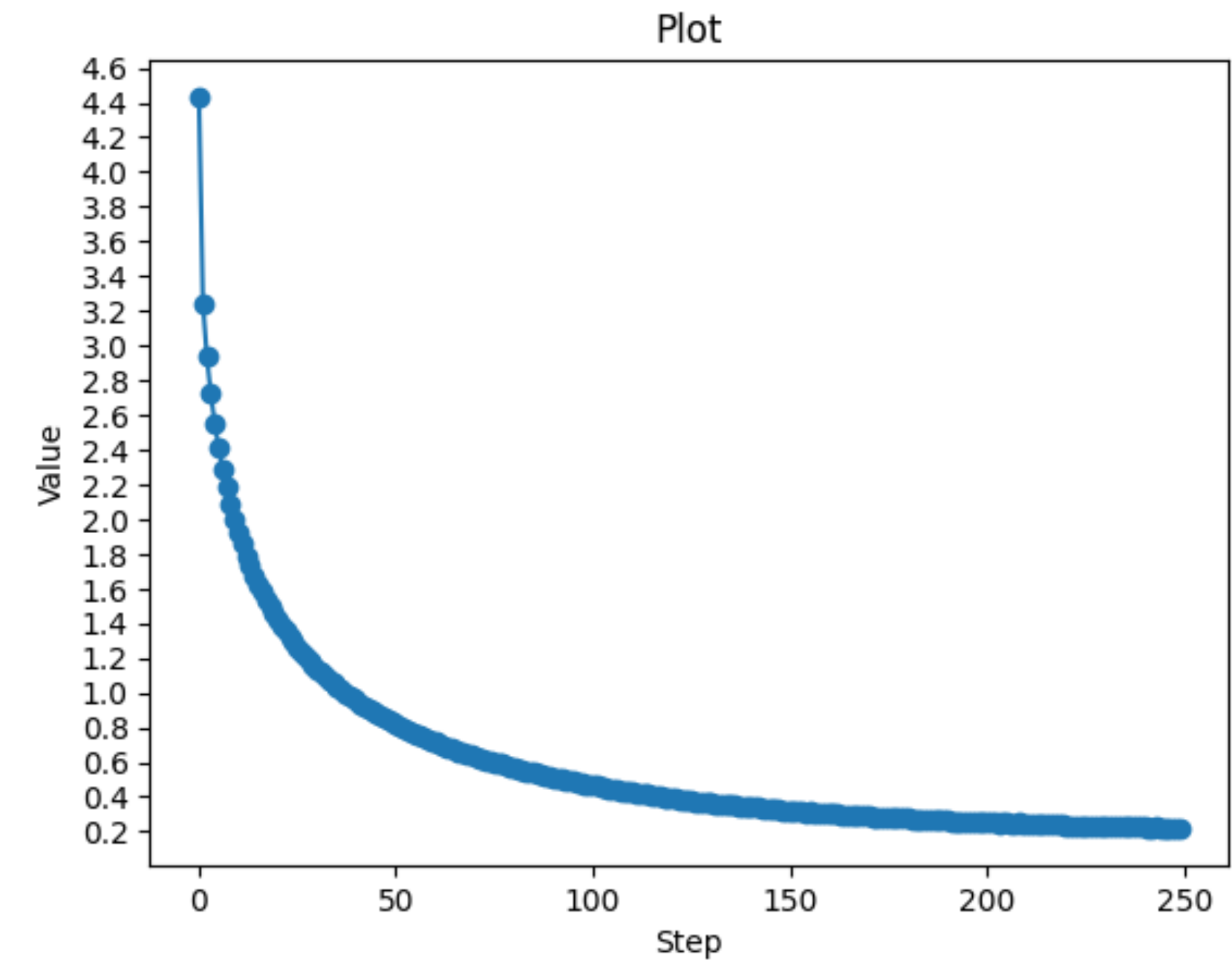
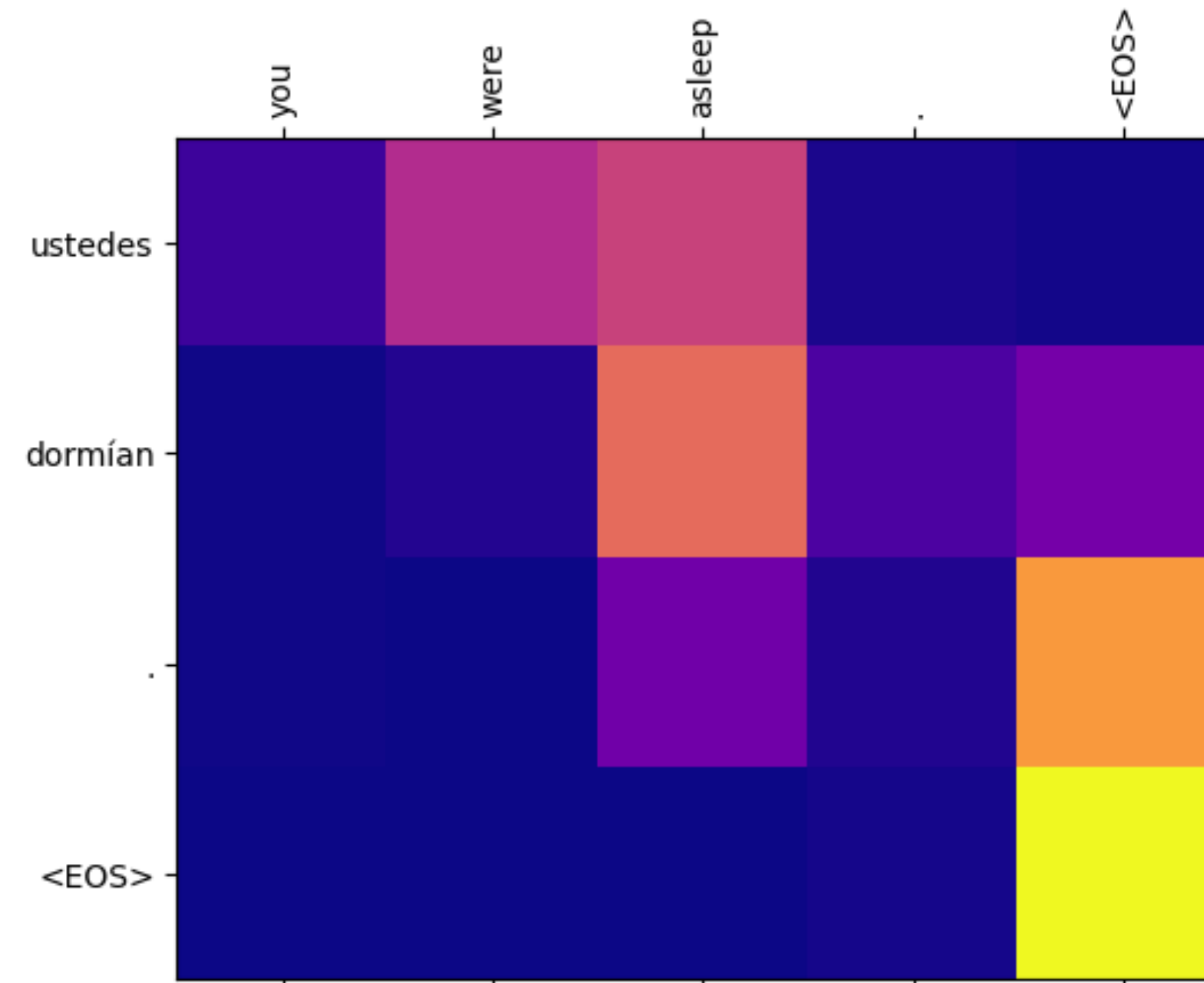
input: are you drunk ?
correct: estás embriagado ?
output: estás borracho ?

input: i study psychology .
correct: estudio psicología .
output: estudio psicología .

input: you were asleep .
correct: ustedes dormían .
output: ustedes dormían .

input: here take this .
correct: toma coge esto .
output: toma coge esto .

input: don t talk .
correct: no hables .
output: no hables .



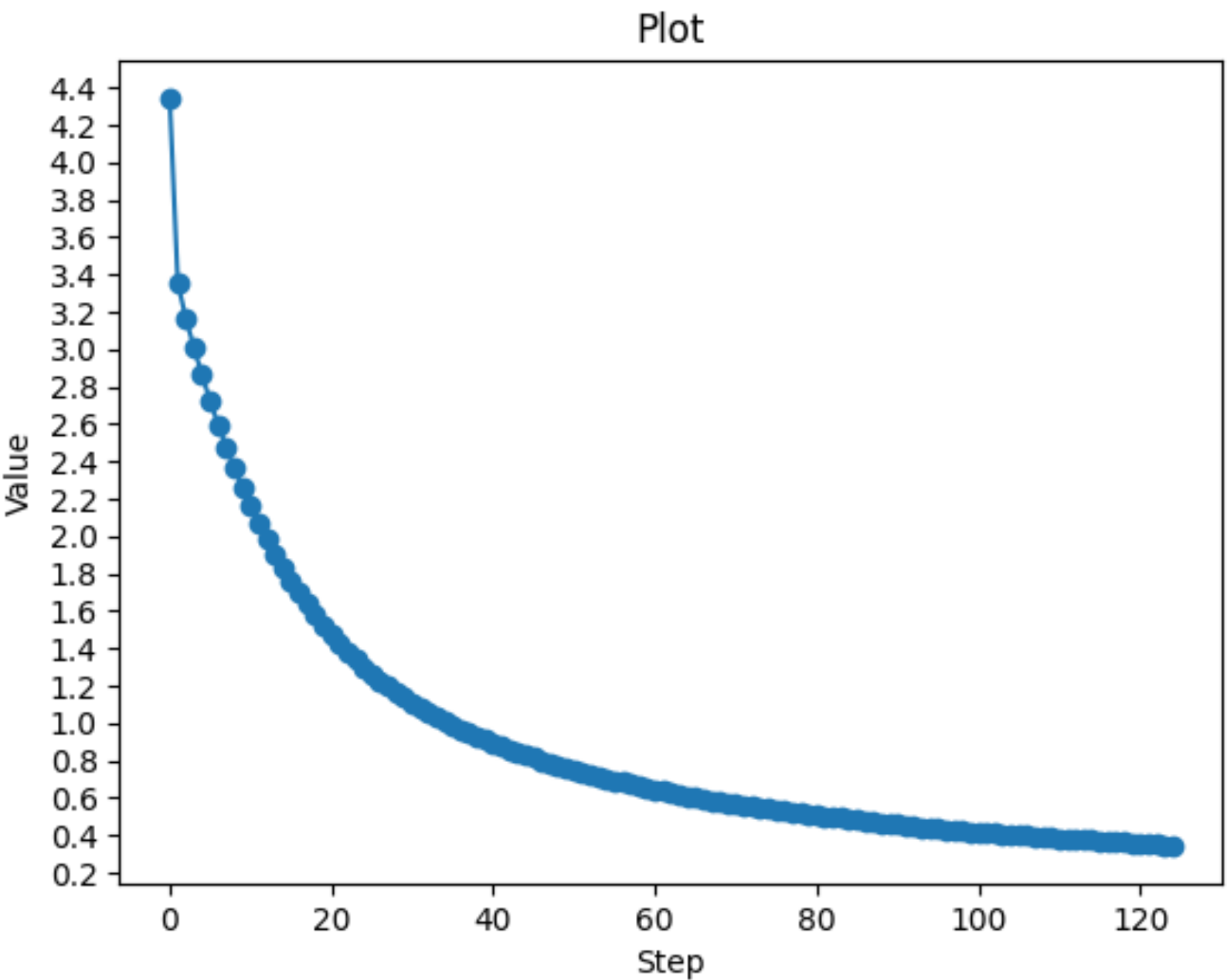
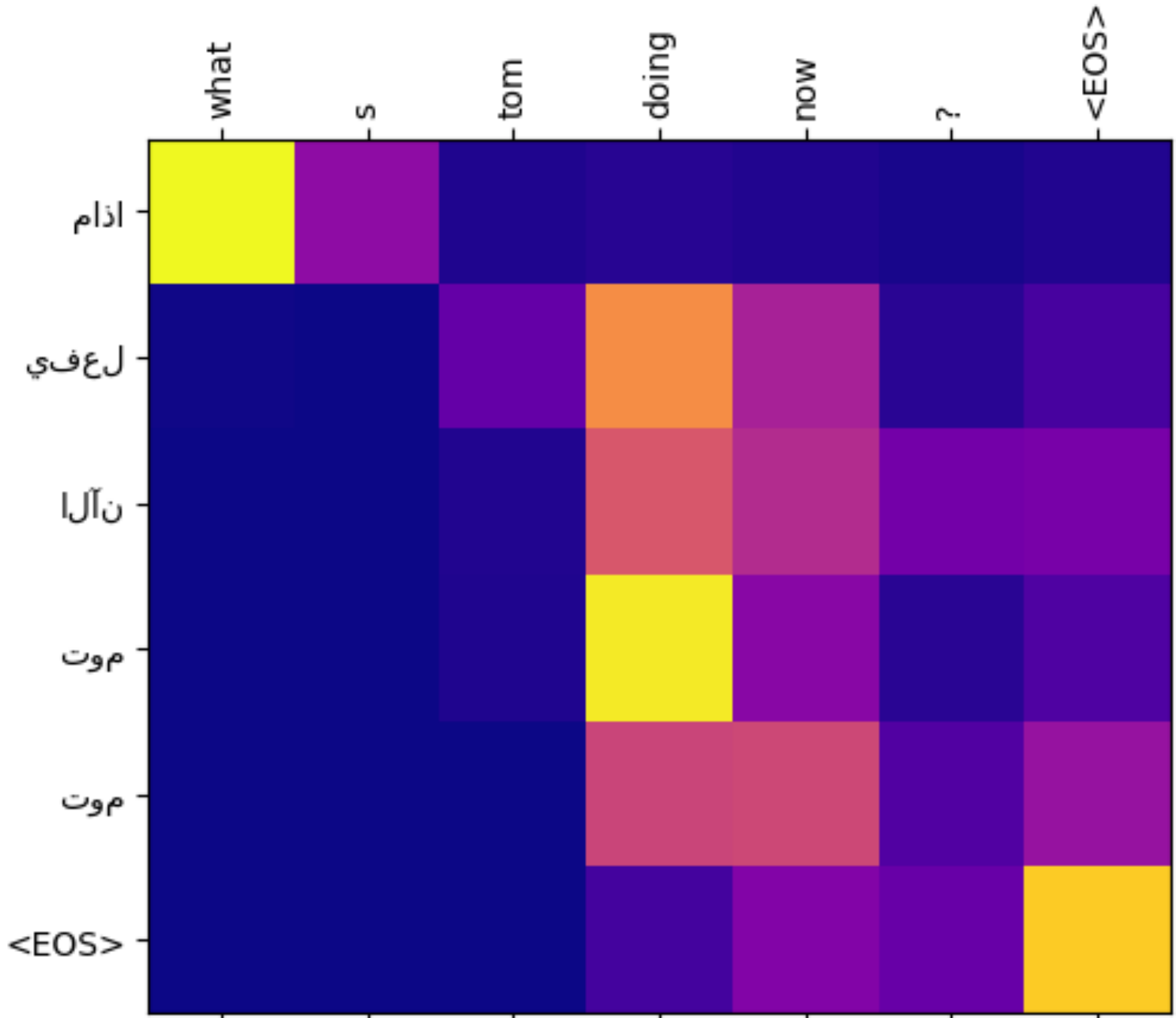
input: we ve arrived .
correct: ها قد وصلنا .
output: لقد وصلنا منذ سنة .

input: it is raining hard .
correct: إنها تمطر بشدة .
output: إنها تمطر بشدة .

input: please write with a pen .
correct: من فضلك اكتب بقلم حبر .
output: من فضلك اكتب بقلم حبر .

input: What's Tom doing now?
correct: ماذا يفعل الآن توم
output: ماذا يفعل الآن توم توم

input: why did you disobey my order ?
correct: لم عصيت أمري
output: لم عصيت أمري



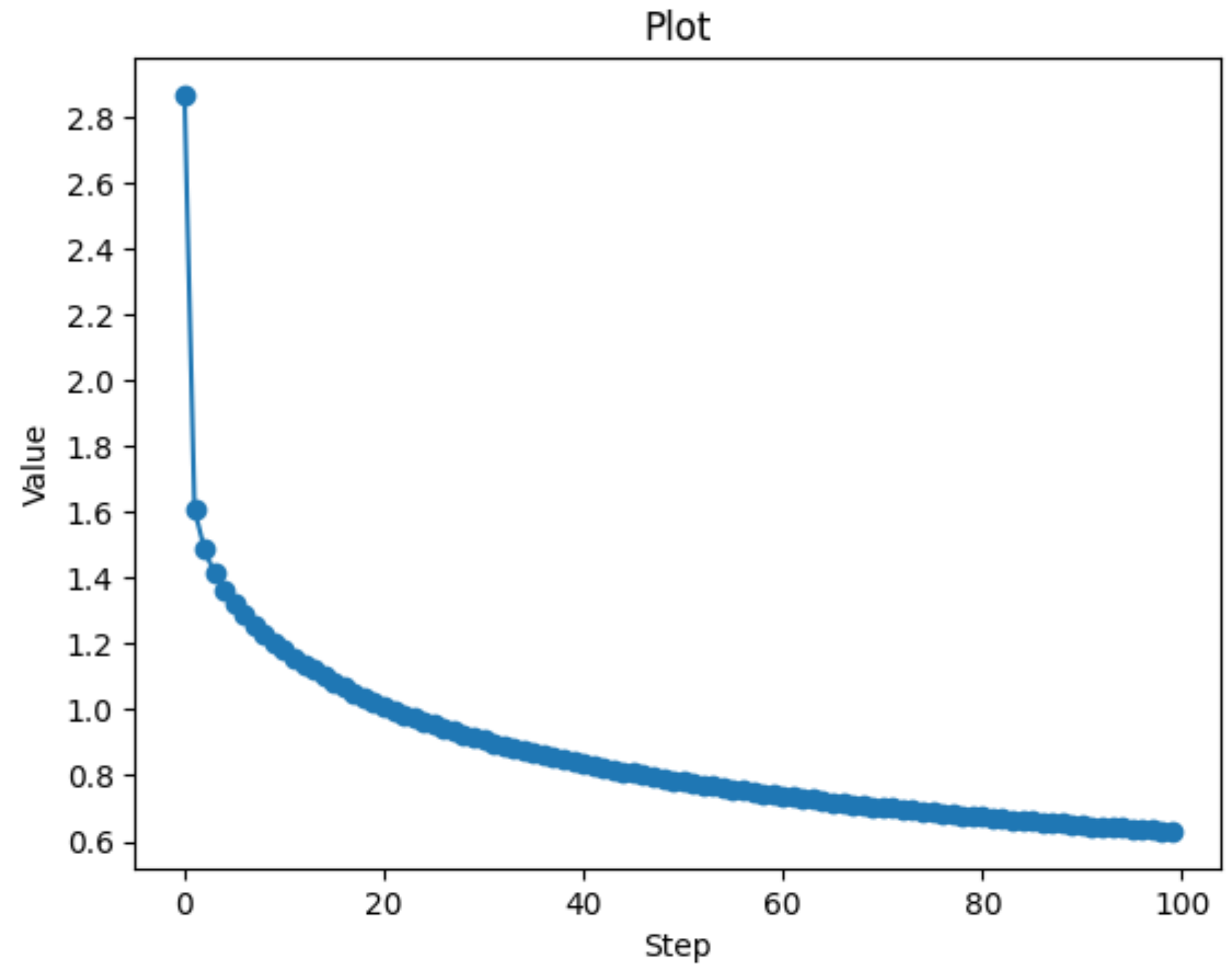
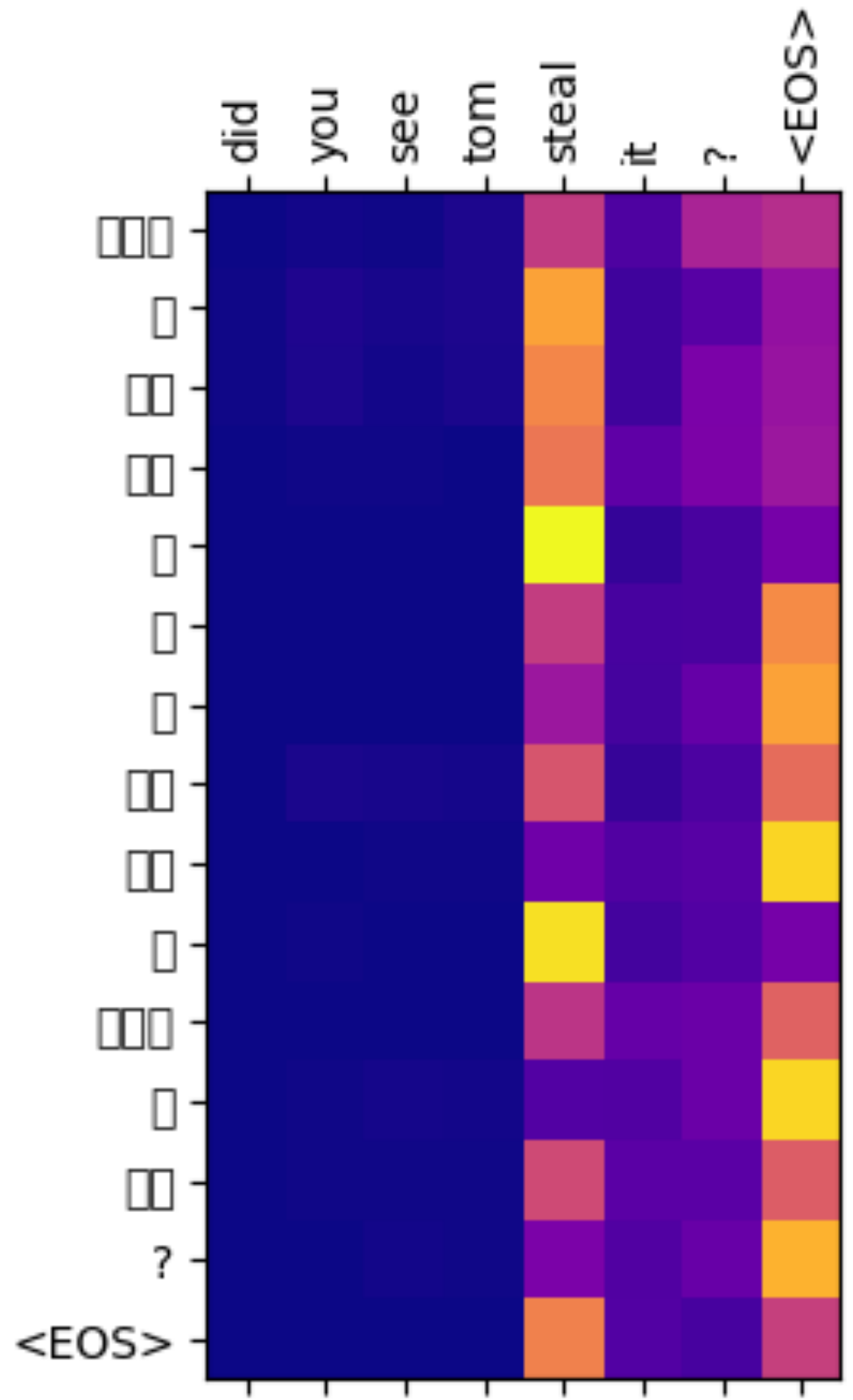
input: why don t we go and see the cherry blossoms ?
correct: ह न म मन उन ज न ह न ?
output: ह ज त म अब

input: did you see tom steal it ?
correct: क तप ई ल टमल च र क द ख न भय ?
output: टमस ग सम झन न ल ई सम झन न टमल ई सम ?

input: i want to be your husband .
correct: म त म र श र म न बन न च हन छ
output: म त म र ज न च हन छ

input: put this in your backpack .
correct: यसल ई आफ न झ ल म र ख न ह स
output: ह म न टमल ई ब झ

input: i am eighteen years old .
correct: म अठ र वर षक ह
output: म ध र भए



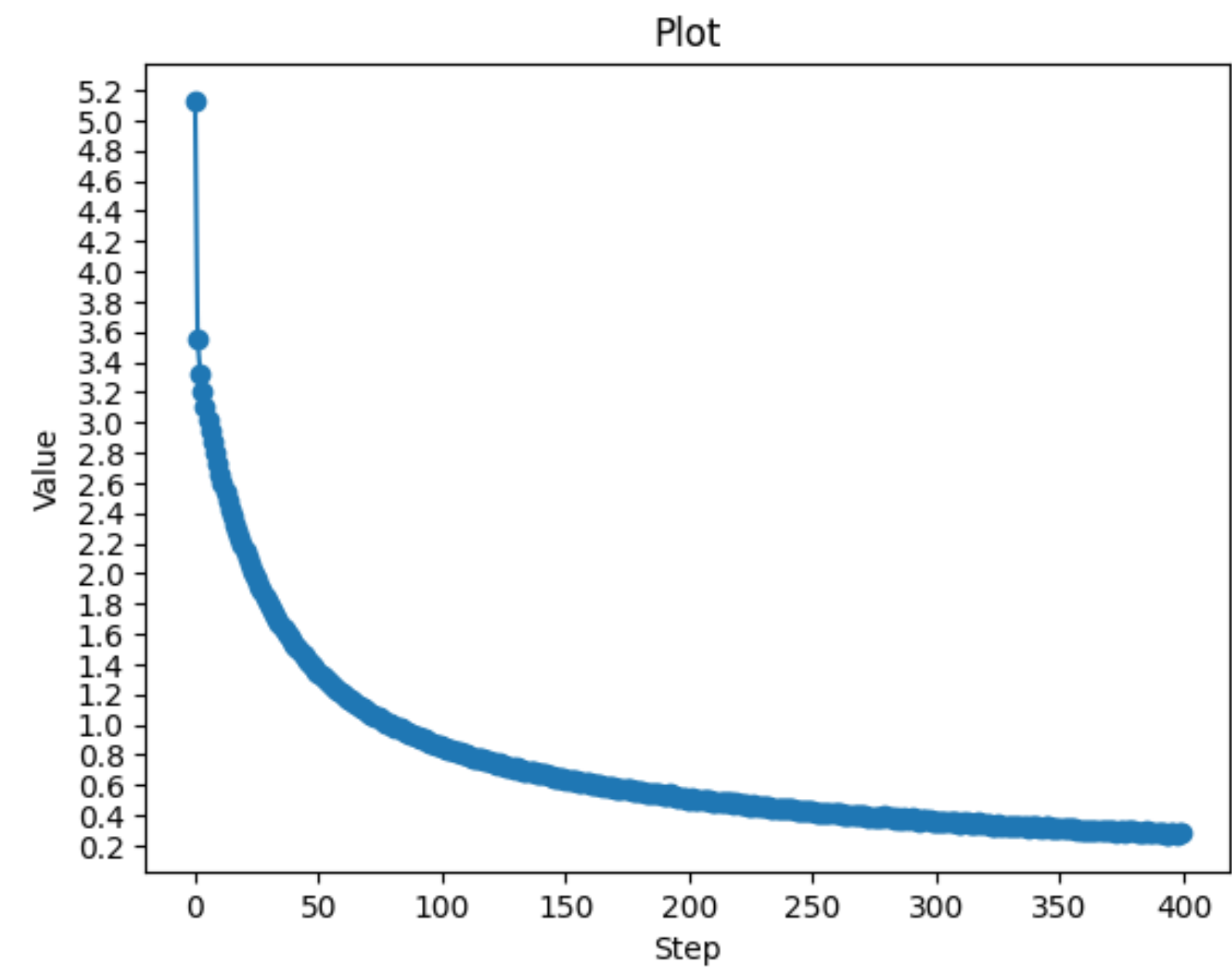
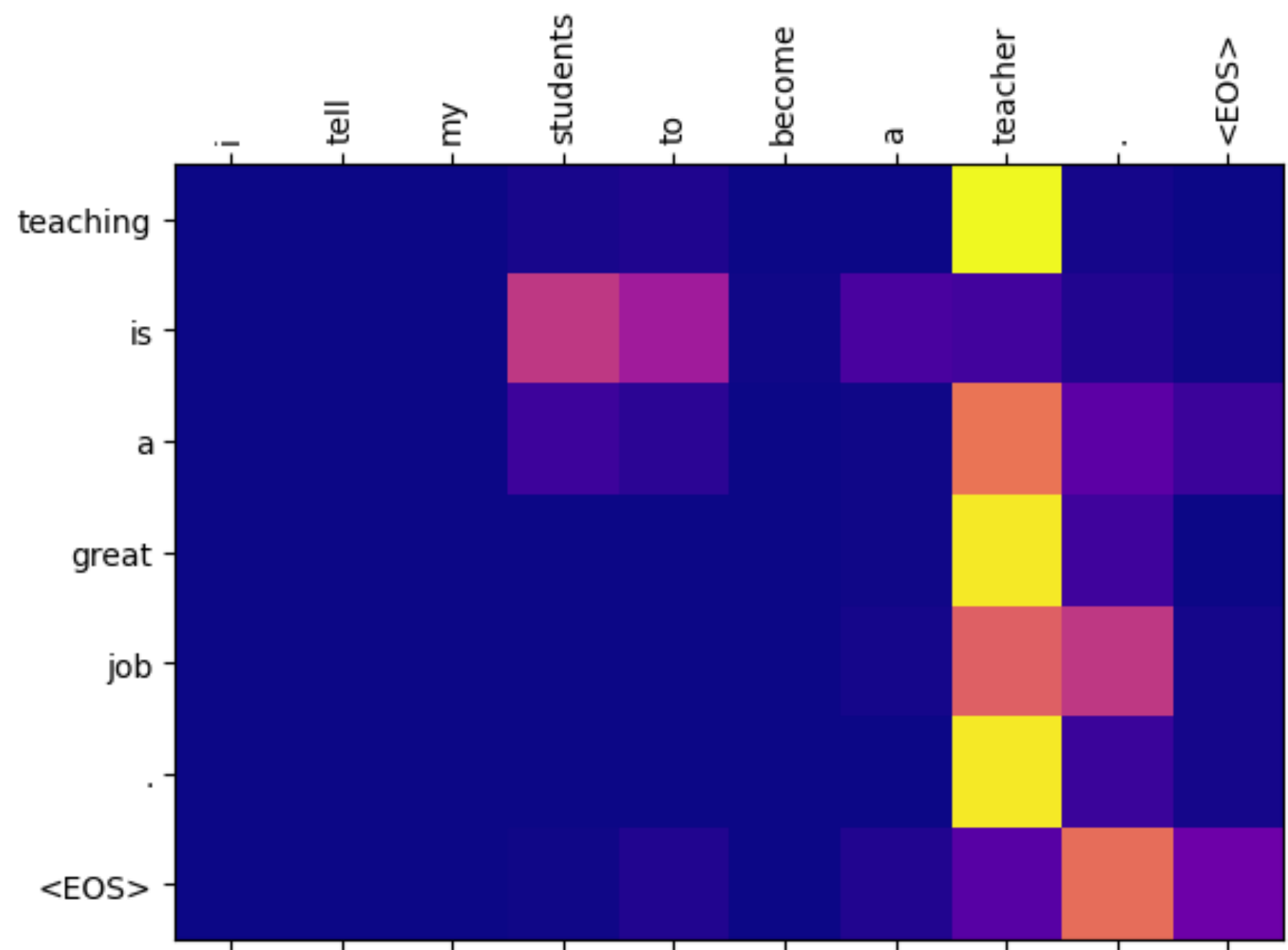
input: that driver is crazy .
correct: i can t believe he hasn t crashed .
output: i can t believe he hasn t crashed .

input: i ll say .
correct: i drove around for half an hour .
output: i drove around for half an hour .

input: i tell my students to become a teacher .
correct: teaching is a great job .
output: teaching is a great job .

input: then why were you looking at her ?
correct: i was looking at something else .
output: i was looking at something else .

input: what if i don t like her ?
correct: then you don t date her again .
output: then you don t date her again .



rule_generator.py

Implication

C -> E; E -> A; C \t A

Boolean

0 AND 1 \t 0

Multistep-boolean

1 AND NOT 0 OR NOT 1 \t 1



D -> F; F -> B; B -> A; D A
F -> E; E -> A; A -> D; F D
F -> D; D -> C; C -> B; F B
NOT 1 AND 0 OR NOT 0 1
NOT 0 AND 0 OR 0 AND NOT 1 0
NOT 1 AND 1 OR NOT 0 AND NOT 1 0
F -> D; D -> B; F B
0 AND NOT 0 OR NOT 0 AND 1 1
1 AND NOT 1 OR NOT 1 AND NOT 0 0
B -> C; C -> D; B D
NOT 0 AND 1 OR 1 AND NOT 1 1
D -> E; E -> C; D C
F -> E; E -> D; F D
NOT 1 AND 1 OR NOT 0 AND 0 0
A -> C; C -> D; D -> E; A E
A -> D; D -> F; F -> B; A B

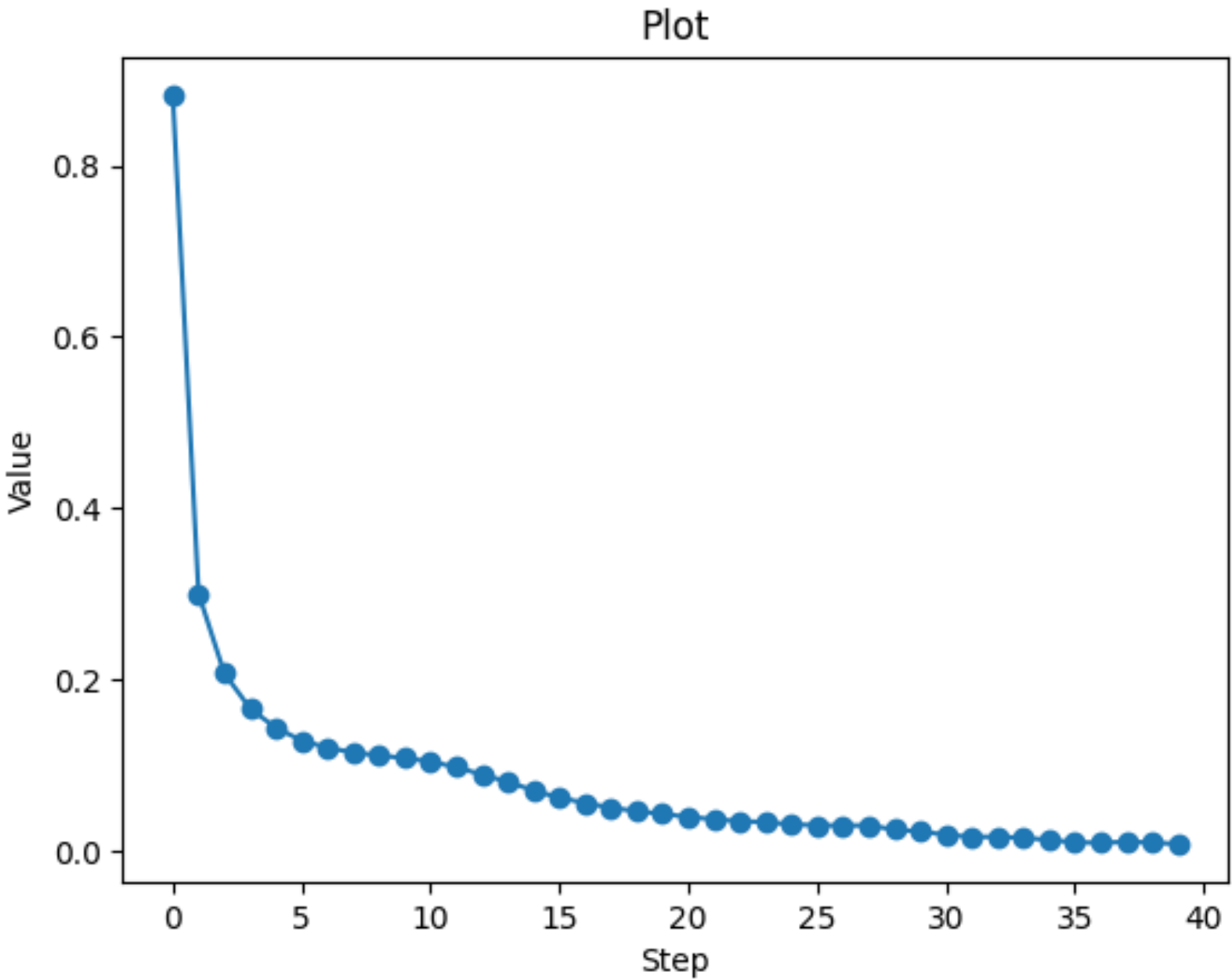
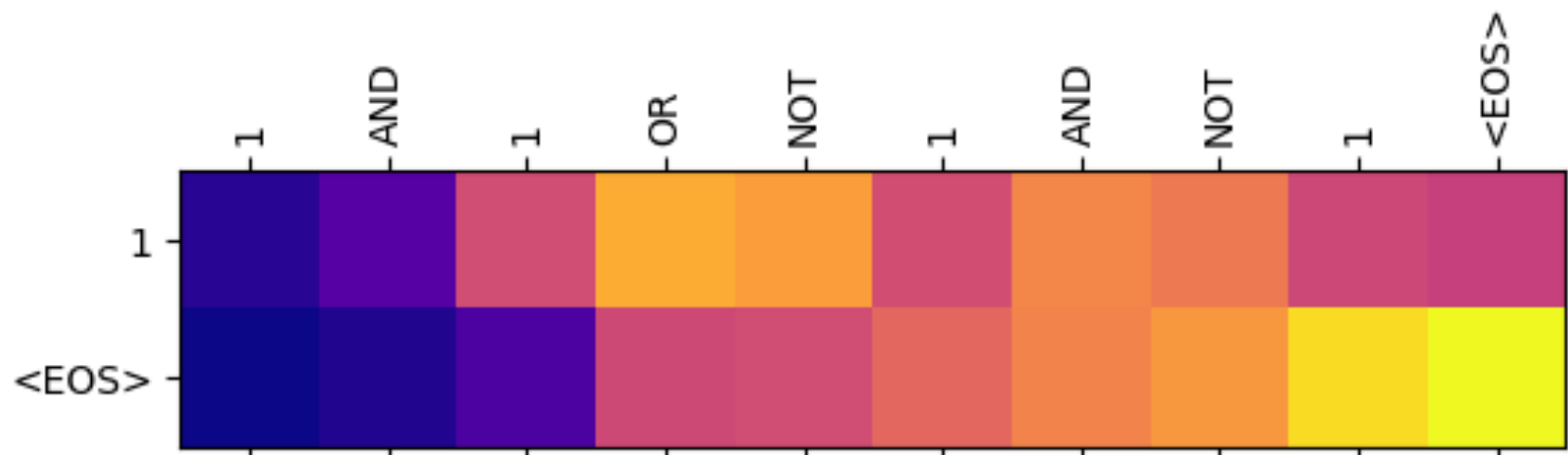
input = 1 AND 1 OR NOT 1 AND NOT 1
correct = 1
output = 1

input: f > b ; b > d ; d > c ; f
correct: c
output: c

input: c > f ; f > e ; c
correct: e
output: e

input: 1 AND NOT 0 OR NOT 0
correct: 1
output: 1

input: d > b ; b > c ; d
correct: c
output: c



Datasets

